Riemannian Optimization for Distance-Geometric Inverse Kinematics

Filip Marić, Matthew Giamou, Adam W. Hall, Soroush Khoubyarian, Ivan Petrović, and Jonathan Kelly

***Abstract*—Solving the inverse kinematics problem is a fundamental challenge in motion planning, control, and calibration for articulated robots. Kinematic models for these robots are typically parameterized by joint angles, generating a complicated mapping between the robot configuration and the end-effector pose. Alternatively, the kinematic model and task constraints can be represented using invariant distances between points attached to the robot. In this article, we formalize the equivalence of distance-based inverse kinematics and the distance geometry problem for a large class of articulated robots and task constraints. Unlike previous approaches, we use the connection between distance geometry and low-rank matrix completion to find inverse kinematics solutions by completing a partial Euclidean distance matrix through local optimization. Furthermore, we parameterize the space of Euclidean distance matrices with the Riemannian manifold of fixed-rank Gram matrices, allowing us to leverage a variety of mature Riemannian optimization methods. Finally, we show that bound smoothing can be used to generate informed initializations without significant computational overhead, improving convergence. We demonstrate that our inverse kinematics solver achieves higher success rates than traditional techniques and substantially outperforms them on problems that involve many workspace constraints.**

***Index Terms*—Computational geometry, kinematics, motion and path planning, Riemannian optimization.**

Manuscript received July 30, 2021; accepted October 7, 2021. Date of publication December 1, 2021; date of current version June 7, 2022. This article was recommended for publication by Associate Editor R. Murrieta-Cid and Editor E. Yoshida upon evaluation of the reviewers' comments. This work was supported in part by the European Regional Development Fund under Grant KK.01.1.1.01.0009 (DATACROSS) and in part by the Natural Sciences and Engineering Research Council of Canada. The work of Jonathan Kelly was supported by the Canada Research Chairs Program. *(Filip Marić and Matthew Giamou contributed equally to this work.) (Corresponding author: Filip Marić.)*

Filip Marić is with the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto Institute for Aerospace Studies, Toronto, ON M3H 5T6, Canada, and also with the Laboratory for Autonomous Systems and Mobile Robotics, Faculty of Electrical Engineering and Computing, University of Zagreb, 10000 Zagreb, Croatia (e-mail: filip.maric@robotics.utias.utoronto.ca).

Matthew Giamou, Soroush Khoubyarian, and Jonathan Kelly are with the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto Institute for Aerospace Studies, Toronto, ON M3H 5T6, Canada (e-mail: matthew.giamou@mail.utoronto.ca; soroush.khoubyarian@mail.utoronto.ca; jkelly@utias.utoronto.ca).

Adam W. Hall is with the Space and Terrestrial Autonomous Robotic Systems Laboratory and the Dynamic Systems Laboratory, University of Toronto Institute for Aerospace Studies, Toronto, ON M3H 5T6, Canada (e-mail: adam.hall@robotics.utias.utoronto.ca).

Ivan Petrović is with the Laboratory for Autonomous Systems and Mobile Robotics, Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb 10000, Croatia, and also with the University of Toronto Institute for Aerospace Studies, Toronto ON M3H 5T6, Canada (e-mail: ivan.petrovic@fer.hr).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TRO.2021.3123841.

Digital Object Identifier 10.1109/TRO.2021.3123841

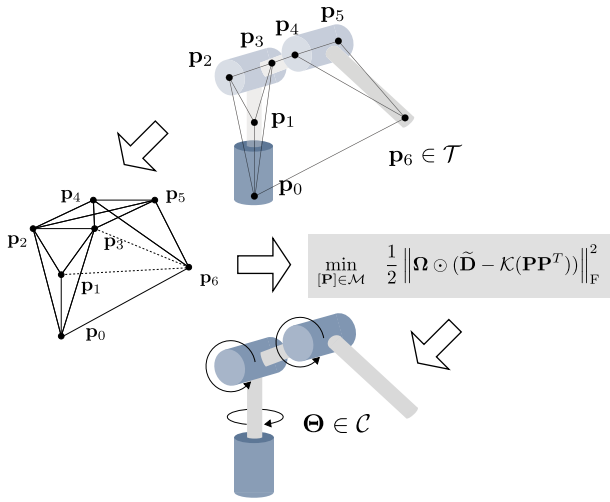

Fig. 1. Overview of the proposed algorithm. A goal end-effector position $\mathbf{p}_6$ is defined for a three-DOF robotic manipulator (top); the IK problem is to find the corresponding joint angles $\mathbf{\Theta}$. Our method uses the matrix $\tilde{\mathbf{D}}$ of distances between points $\mathbf{P}$ common to all feasible IK solutions to define an incomplete graph whose edges are weighted by known distances. Then, we apply EDM completion with the known distance selection matrix $\mathbf{\Omega}$ to recover the weights corresponding to the unknown edges, solving the IK problem.

I. Introduction

ARTICULATED robots consist of actuated revolute joints connected by rigid links. A significant portion of the difficulty associated with performing a specific task involves finding joint angles that achieve a desired end-effector pose. Identifying a set of joint angles or a *configuration* that reaches the desired goal pose(s) of one or more end-effectors is known as the *inverse kinematics* (IK) problem [1]. In general, this problem cannot be solved analytically and admits an infinite number of solutions for robots with redundant degrees of freedom (DOFs). Therefore, most approaches resort to numerical methods that solve constrained local optimization problems over joint angles. This leads to constraints on end-effector and link poses that manifest as nonlinear expressions involving joint angles and kinematic parameters such as link lengths. Without an informed or lucky initial guess, local search algorithms may converge to local minima, leading to inefficient performance or failure to find a sufficiently accurate solution.

Many problems can be expressed using points and their relative distances [2]. In the case of IK, a geometrically intuitive alternative to joint angle parameters is obtained by considering points attached to the body of the robot [3]. The positions of

these points on the robot and their relative distances can be used to describe the kinematic model and end-effector goal poses within an IK formulation [4], [5]. This approach unifies the domain and codomain (i.e., the configuration and task spaces) of the kinematic model, eliminating trigonometric constraints that appear when using joint angles. In contrast to an angle-based parameterization, the search space is not restricted to feasible configurations, but to arbitrary point "conformations" in Euclidean space. The convergence of such approaches is hindered by a large and highly redundant search space, in part because the point set can be rotated and translated without affecting the distance constraints. A more compact representation is obtained by modeling the robot using *distances* as variables [6], [7]. While this avoids the redundancy induced by the invariance of the points to rigid transformations, the search space also includes points in higher dimensions, as there is no guarantee that a given set of distances can be realized as a collection of points in 2-D or 3-D Euclidean space.

In this article, we explore IK from a *distance geometry* perspective, revealing an equivalence between IK and the general distance geometry problem (DGP) [8]. By formalizing this equivalence for a large class of robots comprised of planar (i.e., 2-D), spherical, and revolute joints, we are able to connect IK to a rich literature of DGP solutions based on low-rank matrix completion [9]. We find solutions using the method introduced by Mishra *et al.* [10], where a Euclidean distance matrix (EDM) [2] is parameterized with the manifold of fixed-rank Gram matrices [11], maintaining the advantages of a relaxed search space with fixed dimensionality and reduced redundancy. Additionally, we show that bound smoothing [12] can be used to generate informed initializations, further improving convergence. Our main contributions are as follows.

1) We present a distance-geometric formulation of IK for a variety of robot types and prove its equivalence to traditional angle-based IK.
2) We extend this formulation by systematically incorporating common configuration and workspace constraints.
3) We show that our IK formulation can be solved using generic Riemannian optimization methods for low-rank matrix completion.
4) We demonstrate that informed initializations can be generated using the bound smoothing method from distance geometry.

Finally, we provide a free and open-source Python implementation of our algorithms and simulation experiments, which empirically show the effectiveness of our approach on a variety of robot models.

Section II briefly reviews existing IK literature and the application of distance geometry to practical problems. Section III covers the relevant background material on distance geometry, elucidating the connection between Euclidean distance matrices, low-rank matrix completion, and Riemannian optimization. Section IV begins with an introduction to the IK problem and its relevant terminology. Sections IV-A–IV-C introduce distance-geometric IK formulations for a variety of robots and constraints (e.g., end-effector poses and joint limits), while Section IV-D formally proves their equivalence to the DGP. Section V gives a detailed description of our IK algorithm and the bound smoothing procedure that can be used to find an informed initialization. Section VI contains extensive experimental results for several commercial manipulators and a variety of complex high-DOF mechanisms. Finally, Section VII concludes this article and provides a discussion of limitations and potential future work. Compared to typical IK formulations based on joint angles, experimental results indicate that our method often achieves higher success rates and faster convergence and outperforms benchmark algorithms when many workspace constraints are present.

II. Related Work

We begin with a discussion of existing theory and algorithms for characterizing and solving IK problems. Since our main contribution is a principled application of distance geometry to IK, we proceed with a review of the distance geometry literature, with a particular focus on its (previously) limited intersection with IK.

A. Inverse Kinematics

Due to its widespread use in areas such as robotics [13] and computer graphics [14], IK remains an active research area with an abundance of relevant literature that we can only briefly summarize here. IK is typically formulated with a set of trigonometric equations and inequalities that constrain the kinematic model configuration to achieve a desired end-effector position or pose [1]. It is well established that kinematic chains with up to six DOFs admit a finite number of configurations that satisfy these constraints for feasible end-effector poses [15]. Moreover, the entire solution set can be determined analytically [16], [17] and even generated in an automated manner using the *IKFast* algorithm [18]. Unfortunately, in many cases of interest, there exist redundant DOFs or multiple end-effectors, rendering an analytical approach infeasible.

Heuristic methods can be used to efficiently solve IK problems for a limited class of robots. The *cyclic coordinate descent* (CCD) algorithm [19] iteratively adjusts joint angles using simple geometric expressions, resulting in very low computational overhead and making it useful in real-time applications. Similarly, the *triangulation* algorithm from [20] incurs an even lower computational overhead than CCD, providing global convergence guarantees within a predetermined number of calculations for robots comprised of joints with unconstrained axes of rotation. While highly efficient, both methods require additional modifications and engineering in order to be adapted to robots with multiple end-effectors or with joint limits. The algorithms developed by Han and Rudolph apply unique parameterizations to IK problems over spherical linkages [7], [21]. These parameterizations reveal a piecewise-convex structure that simplifies the solution, but the approach does not apply to generic revolute manipulators and cannot readily incorporate simple constraints like joint limits. Recently, Aristidou *et al.* have introduced FABRIK [22], [23], a heuristic solution to the IK problem, which uses iterative forward–backward passes over joint positions to quickly produce high-quality solutions for a

variety of robot models. Because of its speed and general applicability, we implement FABRIK as a benchmark for comparison with our algorithm in Section VI.

IK is often formulated as a local optimization problem over joint angles and solved using unconstrained or bounded nonlinear programming methods such as L-BFGS-B [24] or SQP [25]. These methods have robust theoretical underpinnings [26] and can approximately support a wide range of constraints through the addition of penalties to the cost function [27]. However, the highly nonlinear nature of the problem makes them susceptible to local minima, often requiring multiple initial guesses before returning a global minimum, if at all. In robotics, many such methods belong to the family of closed-loop IK (CLIK) techniques [1], where the kinematic Jacobian (pseudo)inverse is used to apply differential kinematics in a closed-loop fashion, emulating a feedback control problem [28]. Major advantages of CLIK methods include their ease of implementation and a variety of extensions providing numerical robustness [29] and efficient incorporation of secondary objectives through redundancy resolution [30]. However, alongside the convergence issues commonly encountered with first-order local optimization, CLIK methods often have problems with singularities [31] due to their reliance on the kinematic Jacobian. In Section VI, we compare our algorithm to the IK solver recently introduced by Erleben and Andrews [32], who show that second-order methods with exact Hessian matrices have improved convergence properties.

Recently, several optimization approaches have been introduced that forgo the standard joint angle parameterization in favor of models based on Cartesian coordinates (also known as *natural coordinates* [3]). Dai *et al.* [33] use a piecewise-convex relaxation of the $\mathrm{SO}(3)$ group together with a set of points on the robot to formulate the constrained IK problem as a mixed-integer linear program. Their formulation can detect infeasible problems and provide approximate solutions to feasible problems, at the cost of a computationally intensive solution method. Yenamandra *et al.* [34] use a similar relaxation to formulate IK as a semidefinite program. Blanchini *et al.* [4], [35] treat points on a rigid manipulator as virtual masses in a potential field, leading to "minimum energy" solutions to convex formulations of planar and spherical IK. Le Naour *et al.* [5] formulate IK as a nonlinear program over interpoint distances, showing that solutions can be recovered for unconstrained articulated bodies. While our kinematic model is based on interpoint distances, our approach differs from previous work by encompassing a larger class of robots and allowing for constraints such as symmetric joint limits and spherical obstacle avoidance. Moreover, we provide a comparison with both heuristic and nonlinear optimization approaches, demonstrating that our proposed solution method provides a benchmark for IK problems.

B. Distance Geometry

The theory of distance geometry plays an important role in the development of computational methods for analyzing problems defined using interpoint distances [8]. This elegant theoretical framework is often applied to solve a diverse set of problems spanning computational chemistry [12], signal processing [36], and acoustics [2]. Liberti *et al.* [8] present a detailed taxonomy of DGPs, which can be collectively described as completing a partially connected graph of interpoint distances. When a high degree of connectivity is present (i.e., most distances are known), the classical multidimensional scaling algorithm [37] is often used. The EMBED algorithm models the problem of molecular conformation using distances, providing bounds on unknown distances using *bound smoothing* [12] and iteratively finding solutions for smaller problem instances. Larger problem instances that satisfy some additional assumptions can be solved with a branch-and-prune strategy [38]. Convex relaxations of the DGP have been coupled with semidefinite programming methods in both chemistry and sensor network localization [39], [40].

Known distances can be arranged in an incomplete EDM [2] of rank at most $K + 2$, where K is the dimension of the Euclidean space. In many applications, the DGP can be solved by determining the unknown EDM entries using a low-rank *matrix completion* approach [9]. Recently, numerical optimization methods based on results from Riemannian geometry have been utilized to efficiently perform EDM completion [41]. Mishra *et al.* solve the EDM completion problem using a Riemannian trust-region (RTR) method by parameterizing the EDM with elements of a quotient manifold invariant to orthogonal transformations [10]. Nguyen *et al.* [42] present a similar approach, that finds solutions to the problem of sensor network localization using a Riemannian conjugate gradient method on a manifold representation of EDMs.

Many problems pertaining to active structures such as robots also admit purely distance-based descriptions [43]. Porta *et al.* [6] relate the IK problem to EDM completion for several common classes of manipulators and leverage an algebraic approach to find configurations reaching a desired end-effector pose. For general systems of kinematic and geometric constraints, Porta *et al.* [44] apply a complete but computationally expensive branch-and-prune solver that iteratively eliminates regions of the solution space using geometric techniques. Our recent work [45] applies a convex *sum-of-squares* (SOS) relaxation [46], [47] to a distance-geometric formulation of IK, exploiting theoretical properties that guarantee a globally optimal solution for many problem instances. Moreover, we previously showed that kinematic constraints induce an inherently sparse structure that can be used to significantly reduce the computational burden usually associated with the SOS approach (and other convex relaxation-based methods).

In this article, we use the Riemannian optimization method in [10] to solve distance-geometric formulations of the IK problem. We model the motion constraints of individual joint–link pairs by defining distances between key points in the structure of the robot in a manner similar to [6], which allows us to set constraints on end-effector poses and joint angles by limiting the associated distances to a predetermined range. In contrast to [6], the numerical optimization approach makes our algorithm suitable for a more general class of robots with redundant DOFs. Crucially, we use the DGP as a mathematical basis for our approach and derive simple and inclusive criteria for the

compatibility of a robot mechanism with our method. Finally, we show that bound smoothing [12] can be used as an effective initialization method that significantly improves convergence.

III. Background

In this section, we state the core DGPs that serve as a theoretical foundation for the proposed IK formulation. By analyzing well-known identities for representing collections of points in the form of Euclidean distance matrices [2], we arrive at a DGP formulation that is based on low-rank matrix completion [48]. Finally, we summarize the optimization-based EDM completion approach introduced in [10], which we use to solve the problems presented herein.

A. Distance Geometry

The fundamental problem of distance geometry, as stated in [8], is as follows.

Problem 1 (DGP): Given an integer $K > 0$, a set of vertices V, and a simple undirected graph $G = (V, E)$ whose edges $\{u, v\} \in E$ (where $u, v \in V$) are assigned nonnegative weights

$$\{u, v\} \mapsto d_{u,v} \in \mathbb{R}_+$$

find a function $p : V \to \mathbb{R}^K$ such that the Euclidean distances between pairs match the assigned weights

$$\forall \{u, v\} \in E\,, \quad \|p(u) - p(v)\| = d_{u,v}. \tag{1}$$

The function $p : V \to \mathbb{R}^K$ is also known as a *realization* of the graph G. Any realization p of G maps all the vertices in V to a collection of points $\mathbf{P} \in \mathbb{R}^{|V| \times K}$, where each row is the position $\mathbf{p}_u = p(u) \in \mathbb{R}^K$ of the point corresponding to vertex $u \in V$.

In some cases, we may wish to reduce the number of possible realizations by constraining a subset of interpoint distances (i.e., edge weights) to some interval. Consequently, we can extend Problem 1 such that the edges in E are weighted by positive intervals, resulting in the more general *interval DGP* [8]:

Problem 2 (Interval DGP): Given an integer $K > 0$ and a simple undirected graph $G = (V, E)$ whose edges $\{u, v\} \in E$ are weighted by intervals

$$\{u, v\} \mapsto [d^-_{u,v}, d^+_{u,v}] \subseteq \mathbb{R}_+$$

find a realization in $\mathbb{R}^K$ such that Euclidean distances between pairs belong to the edge intervals

$$\forall \{u, v\} \in E\,, \quad \|p(u) - p(v)\| \in [d^-_{u,v}, d^+_{u,v}]. \tag{2}$$

Note that for all $e = \{u, v\} \in E$, the notation for Problem 2 supports unconstrained or missing distances ($d^-_e = 0$, $d^+_e \to \infty$), as well as the equality constraints found in Problem 1 ($d^-_e = d^+_e$).

In this article, we refer to both Problems 1 and 2 as the DGPs, where the specific formulation can be inferred from the presence or absence of distance intervals on the edge weights of G. If a realization that satisfies an instance of the DGP exists, the corresponding collection of points $\mathbf{P}$ can be arbitrarily translated, rotated, and reflected such that the distance constraints still hold [2]. This defines the equivalence class $[\mathbf{P}]$ of (14). Additionally, there may be multiple equivalence classes $[\mathbf{P}]$ that correspond to distinct solutions to Problem 1 or 2. We will make use of this distinction in Section IV-D.

B. Euclidean Distance Matrices

Consider a realization of a graph G, obtained by solving Problem 1. By arranging the resulting points in a matrix $\mathbf{P} = [\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_{N-1}]^\mathsf{T} \in \mathbb{R}^{N \times K}$, all interpoint distances $d_{u,v}$ can be determined via the Euclidean norm

$$d_{u,v} = \|\mathbf{p}_u - \mathbf{p}_v\|\,.$$

The product $\mathbf{X} \triangleq \mathbf{P}\mathbf{P}^\mathsf{T}$ is known as the *Gram matrix* and belongs to $\mathcal{S}^N_+$, the set of $N \times N$ symmetric positive-semidefinite matrices. Elements of the Gram matrix can conveniently express squared interpoint distances

$$d^2_{u,v} = \mathbf{X}_{u,u} - 2\mathbf{X}_{u,v} + \mathbf{X}_{v,v} \tag{3}$$

and the full set of squared interpoint distances in (3) can be efficiently calculated using the matrix identity

$$\mathbf{D} = \operatorname{diag}(\mathbf{X})\mathbf{1}^\mathsf{T} + \mathbf{1}\operatorname{diag}(\mathbf{X})^\mathsf{T} - 2\mathbf{X} \tag{4}$$

where $\operatorname{diag}(\mathbf{X})$ is the vector formed by the main diagonal of the Gram matrix [2], and $\mathbf{1}$ is a column vector of ones. The resulting matrix $\mathbf{D}$ is known as an EDM. We use $\mathcal{K}(\mathbf{X})$ to denote the linear operator mapping $\mathbf{X}$ to $\mathbf{D}$, as defined by (4).

Consider the problem of recovering the original collection of points $\mathbf{P}$ from squared interpoint distances in the matrix $\mathbf{D}$. Necessary and sufficient conditions for a matrix to be an EDM can be found in [49]. If $\mathbf{D}$ is an EDM, a Gram matrix that satisfies (4) can be recovered by taking

$$\mathbf{X} = -\frac{1}{2}\mathbf{J}\mathbf{D}\mathbf{J} \tag{5}$$

where $\mathbf{J} = \mathbf{I} - \frac{1}{N}\mathbf{1}\mathbf{1}^\mathsf{T}$ is the so-called geometric centering matrix [2]. Once $\mathbf{X}$ has been recovered, a collection of points $\hat{\mathbf{P}} \in \mathbb{R}^{N \times K}$ can be obtained through the eigenvalue decomposition $\mathbf{X} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\mathsf{T}$ by taking the first K eigenvalues λ_i:[1]

$$\hat{\mathbf{P}}^\mathsf{T} = \left[\operatorname{diag}\left(\sqrt{\lambda_0}, \sqrt{\lambda_1}, \ldots, \sqrt{\lambda_{K-1}}\right), \mathbf{0}_{K \times N-K}\right] \mathbf{U}^\mathsf{T}. \tag{6}$$

While the squared distances of points in $\hat{\mathbf{P}}$ recovered using this procedure match those defined in $\mathbf{D}$ exactly, they are, in general, not equal to the original $\mathbf{P}$. This is due to the fact that interpoint distances are preserved under rigid transformations. In order to recover the absolute positions of the points, at least K points, known as *anchors*, need to have their positions defined *a priori*. These anchors are used to formulate the orthogonal Procrustes problem [2], whose solution is the rotation (or reflection) $\mathbf{R} \in \mathrm{O}(K)$ and translation $\mathbf{t} \in \mathbb{R}^K$ that transform the positions of anchors in $\hat{\mathbf{P}}$ to their predefined positions. This transformation can then be applied to all the points in $\hat{\mathbf{P}}$ to yield the desired set of points $\mathbf{P}$.

[1] Assuming that the recovered realization is K-dimensional, only the first K eigenvalues are nonzero.

C. Euclidean Distance Matrix Completion

As discussed in Section III-A, a collection of points can be described using a graph $G = (V, E)$ that is weighted by interpoint distances. If all interpoint distances are known, the graph is *complete*, meaning that all of its edges and corresponding weights are prescribed. This graph can be compactly represented by the EDM whose elements are

$$\forall \{u, v\} \in E, \quad \mathbf{D}_{u,v} \triangleq d_{u,v}^2 \tag{7}$$

and a realization of the graph can be obtained simply by taking the collection of points recovered via (5) and (6). It follows that the recovered collection of points is in fact a solution of the DGP defined by Problem 1. Conversely, many DGP instances are represented by graphs that only have a subset of edges defined *a priori*, resulting in an EDM with missing elements.

The problem of finding the missing elements in a partially defined EDM is known as the *EDM completion problem* [2], which is strongly NP-hard in general [8]. By defining the symmetric binary matrix $\mathbf{\Omega}$ with elements

$$\mathbf{\Omega}_{u,v} \triangleq \begin{cases} 1, & \text{if } \{u, v\} \in E \\ 0, & \text{otherwise} \end{cases} \tag{8}$$

we arrive at a common statement of the EDM completion problem as low-rank matrix completion

$$\min_{\mathbf{X} \in \mathcal{X}} f(\mathbf{X}) \triangleq \frac{1}{2} \left\| \mathbf{\Omega} \odot (\widetilde{\mathbf{D}} - \mathcal{K}(\mathbf{X})) \right\|_{\mathrm{F}}^2 \tag{9}$$

where $\odot$ is the Hadamard (elementwise) matrix product, the subscript F denotes the Frobenius norm, and $\widetilde{\mathbf{D}}$ is the incomplete distance matrix. Since the workspace dimension K of the robot is known, the Gram matrix $\mathbf{X}$ defined in (3) is constrained to the manifold

$$\mathcal{X} = \left\{ \mathbf{P}\mathbf{P}^{\mathsf{T}} : \mathbf{P} \in \mathbb{R}_*^{N \times K} \right\} \tag{10}$$

which is exactly the manifold of rank-K positive-semidefinite matrices. The NP-hardness of this problem originates from the nonconvex constraint on the rank of $\mathbf{X}$, which can be relaxed in order to obtain a solution using Euclidean local search or *semidefinite programming* [50]. From (10), we see that relaxing the rank constraint expands the search space to collections of points with dimension greater than K. This is fundamentally incompatible with physical problems, for which the dimension of points is fixed. In fact, many interior point methods that solve the resulting convex semidefinite program tend to return a max-rank (and, therefore, potentially nonphysical) solution [51].

We can avoid explicit rank constraints in (9) by using the Burer–Monteiro factorization [52] to define the cost function directly in terms of the points $\mathbf{P} \in \mathbb{R}^{N \times K}$. This results in a nonconvex optimization problem

$$f^* = \min_{\mathbf{P} \in \mathbb{R}^{N \times K}} f(\mathbf{P}\mathbf{P}^{\mathsf{T}}) \tag{11}$$

which reduces the number of variables without changing the global minimum [53]. Following the derivation in [10], the gradient of (9) with respect to $\mathbf{P}$ is defined as

$$\nabla_{\mathbf{P}} f = 4 \left(\mathbf{S} - \operatorname{diag}(\mathbf{S}\mathbf{1}) \right) \mathbf{P} \tag{12}$$

where $\mathbf{S} = \mathbf{\Omega} \odot (\widetilde{\mathbf{D}} - \mathcal{K}(\mathbf{X}))$ and $\operatorname{diag}(\mathbf{S}\mathbf{1})$ is a diagonal matrix formed by the vector $\mathbf{S}\mathbf{1}$.

Second-order optimization methods benefit from an exact analytical expression of the Hessian $\nabla_{\mathbf{P}}(\nabla_{\mathbf{P}} f) = \nabla_{\mathbf{P}}^2 f$. In [54], an analytic expression for the full Hessian of (11) is obtained in an elementwise fashion. However, this expensive computation can be avoided by observing that many optimization methods only require the Hessian-vector product [55] and by making use of the identity

$$D_{\mathbf{Z}}(\nabla_{\mathbf{P}} f) \triangleq \nabla_{\mathbf{P}}^2 f \cdot \mathbf{Z}$$

where $D_{\mathbf{Z}}(\nabla_{\mathbf{P}} f) = \frac{d \nabla f((\mathbf{P} + t\mathbf{Z})(\mathbf{P} + t\mathbf{Z})^{\mathsf{T}})}{dt}$ is the Frechét (directional) derivative of the gradient in the direction $\mathbf{Z}$. We then obtain

$$\begin{aligned} \nabla_{\mathbf{P}}^2 f \cdot \mathbf{Z} = \; & 4\, D_{\mathbf{Z}} \left(\mathbf{S} - \operatorname{diag}(\mathbf{S}\mathbf{1}) \right) \mathbf{P} \\ & + 4 \left(\mathbf{S} - \operatorname{diag}(\mathbf{S}\mathbf{1}) \right) \mathbf{Z}. \end{aligned} \tag{13}$$

Unlike gradient descent, second-order optimization methods feature a superlinear convergence rate, which is useful when highly accurate solutions of (11) are required.

D. Optimization on the Manifold

As stated in Section III-B, interpoint distances are invariant to rigid transformations of the underlying point set. It follows that the problem in (11) is invariant to right-multiplication of the variable $\mathbf{P}$ with orthogonal matrices $\mathbf{Q} \in \mathrm{O}(K)$. This results in nonisolated minima, which have been shown to cause step evaluation issues for second-order methods [11], [56] when close to a solution, rendering the classical result of superlinear convergence void. This issue is circumvented in [11] by considering the set of all equivalence classes of the form

$$[\mathbf{P}] = \left\{ \mathbf{P}\mathbf{Q} | \, \mathbf{Q} \in \mathbb{R}^{K \times K}, \mathbf{Q}^{\mathsf{T}}\mathbf{Q} = \mathbf{I} \right\}. \tag{14}$$

Elements of this set constitute a manifold $\mathcal{M}$, which is the quotient of the set of full-rank $N \times K$ matrices by the orthogonal group $\mathrm{O}(K)$

$$\mathcal{M} \triangleq \mathbb{R}_*^{N \times K} / \mathrm{O}(K). \tag{15}$$

It follows that the overall search space is reduced by reformulating (11) on the manifold $\mathcal{M}$ as

$$\phi^* = \min_{[\mathbf{P}] \in \mathcal{M}} \phi([\mathbf{P}]) \tag{16}$$

where $\phi([\mathbf{P}]) = f(\mathbf{P}\mathbf{P}^T)$. Furthermore, it can be shown that the quotient manifold $\mathcal{M}$ has the structure of a Riemannian manifold [56].

Definition 1 (Riemannian manifold): A Riemannian manifold $\mathcal{M}$ is a smooth manifold equipped with a positive-definite inner product $g_{\mathbf{P}} : T_{\mathbf{P}}\mathcal{M} \times T_{\mathbf{P}}\mathcal{M} \to \mathbb{R}$ on the tangent space $T_{\mathbf{P}}\mathcal{M}$ at each point $\mathbf{P} \in \mathcal{M}$ that varies smoothly from point to point.

Next, we provide an overview of how the EDM completion problem in (16) can be adapted to the Riemannian setting, as described by Mishra *et al.* [10]. We refer the reader to [56] for a detailed treatment of quotient manifolds and their geometry.

Formally, the tangent space $T_{\mathbf{P}}\mathcal{M}$ of a point $\mathbf{P}$ in $\mathcal{M}$ is the space of all tangent vectors $\gamma'(0)$ to curves $\gamma : \mathbb{R} \to \mathcal{M}$, where $\gamma(0) = \mathbf{P}$. The tangent space $T_{\mathbf{P}}\mathcal{M}$ is endowed with the inner product, also known as the Riemannian metric

$$g_{\mathbf{P}}(\mathbf{Z}_1, \mathbf{Z}_2) = \mathrm{Tr}(\mathbf{Z}_1^{\mathsf{T}}\mathbf{Z}_2), \quad \mathbf{Z}_1, \mathbf{Z}_2 \in T_{\mathbf{P}}\mathcal{M} \tag{17}$$

which is the usual metric on $\mathbb{R}^{N\times K}$. We can divide the tangent space into two orthogonal subspaces [11]

$$T_{\mathbf{P}}\mathcal{M} = \mathcal{V}_P\mathcal{M} \oplus \mathcal{H}_P\mathcal{M}$$

where the tangent space to the equivalence classes in (14) is known as the *vertical subspace*

$$\mathcal{V}_P\mathcal{M} = \left\{\mathbf{PQ} | \, \mathbf{Q} \in \mathbb{R}^{K\times K}, \mathbf{Q}^{\mathsf{T}} + \mathbf{Q} = 0\right\} \tag{18}$$

and the orthogonal complement of $\mathcal{V}_P\mathcal{M}$ in $T_P\mathcal{M}$ is known as the *horizontal subspace*

$$\mathcal{H}_{\mathbf{P}}\mathcal{M} = \left\{\mathbf{Z} \in T_{\mathbf{P}}\mathbb{R}_*^{N\times K} | \, \mathbf{Z}^{\mathsf{T}}\mathbf{P} = \mathbf{P}^{\mathsf{T}}\mathbf{Z}\right\}. \tag{19}$$

Given a tangent vector $\mathbf{Z} \in T_{\mathbf{P}}\mathcal{M}$ at a point $\mathbf{P} \in \mathcal{M}$, we can recover the horizontal component from (19) using the *horizontal projection* operator.

Definition 2 (Horizontal projection): The horizontal projection $P_{\mathcal{H}_{\mathbf{P}}\mathcal{M}} : T_{\mathbf{P}}\mathcal{M} \to \mathcal{H}_{\mathbf{P}}\mathcal{M}$ that recovers the horizontal lift $\mathbf{Z}_{\mathcal{H}}$ of the tangent vector $\mathbf{Z} \in T_{\mathbf{P}}\mathcal{M}$ corresponding to the horizontal subspace in (19) is defined as

$$P_{\mathcal{H}_{\mathbf{P}}\mathcal{M}}(\mathbf{Z}) = \mathbf{Z} - \mathbf{PC} \tag{20}$$

where $\mathbf{C}$ is a skew-symmetric matrix solving the Sylvester equation

$$\mathbf{C}\mathbf{P}^{\mathsf{T}}\mathbf{P} + \mathbf{P}^{\mathsf{T}}\mathbf{P}\mathbf{C} = \mathbf{P}^{\mathsf{T}}\mathbf{Z} - \mathbf{Z}^{\mathsf{T}}\mathbf{P}.$$

Using the projection operator $P_{\mathcal{H}_{\mathbf{P}}\mathcal{M}}$, derivatives of the function ϕ (defined on the manifold) are computed from the derivatives of the function f (defined in Euclidean space) [56] by producing the horizontal lift of the Euclidean gradient of f at point $\mathbf{P}$

$$\mathrm{grad}_{\mathbf{P}}\phi = P_{\mathcal{H}_{\mathbf{P}}\mathcal{M}}(\nabla_{\mathbf{P}} f). \tag{21}$$

Similarly, by projecting the directional derivative of the gradient defined in (13), we compute the Hessian-vector product of ϕ from that of f as

$$\mathrm{Hess}_{\mathbf{P}}\phi\,[\mathbf{Z}] = P_{\mathcal{H}_{\mathbf{P}}\mathcal{M}}(D_{\mathbf{Z}}(\mathrm{grad}_{\mathbf{P}}\phi)). \tag{22}$$

Once the geometrically correct derivatives have been produced, the step size is calculated, and the point is moved along a descent direction on the manifold. To ensure the resulting point remains on the manifold, we use the *retraction* operator.

Definition 3 (Retraction): In order to apply a direction of movement in $\mathcal{H}_{\mathbf{P}}\mathcal{M}$ while staying on the manifold $\mathcal{M}$, we use the *retraction* operator, which is defined as

$$R_{\mathbf{P}}(\mathbf{W}) = \mathbf{P} + \mathbf{W}. \tag{23}$$

The projection and retraction operators allow for the adaptation of classic local optimization algorithms to the Riemannian setting [57], [58].

IV. Distance-Geometric Inverse Kinematics

Articulated bodies, such as the robotic manipulator shown in Fig. 1, are composed of revolute joints connected by rigid links. Joint angles can be arranged in a vector $\mathbf{\Theta} \in \mathcal{C}$, where $\mathcal{C} \subseteq \mathbb{R}^n$ is known as the *configuration space*. Analogously, the poses of one or multiple end-effectors constitute the *task space* $\mathcal{T}$. The mapping $F : \mathcal{C} \to \mathcal{T}$, relating joint variables to task space coordinates (e.g., end-effector poses), is known as the *forward kinematics* of a robot. This leads to the accompanying notion of *IK*, which is simply the inverse mapping $F^{-1} : \mathcal{T} \to \mathcal{C}$. Since the mapping F^{-1} is generally not injective (i.e., there are multiple solutions for a single target pose) and cannot be determined analytically, numerical methods are instead used to find a single solution. This leads us to the definition of the central problem in this article.

Problem 3 (IK): Given a task space goal $\mathbf{w} \in \mathcal{T}$, find a configuration $\mathbf{\Theta} \in \mathcal{C}$ such that the forward kinematic mapping $F(\mathbf{\Theta}) = \mathbf{w}$ holds.

Note that the IK problem extends to cases where various constraints on the position of the robot are present. For example, it is often necessary to avoid collision with obstacles in the environment, as well as to respect limits on joint angles. Therefore, while the core problem is relatively simple, it can be extended to include challenging instances that commonly occur in practice. Unlike most approaches that attempt to directly solve Problem 3 in terms of joint variables $\mathbf{\Theta}$, we adopt an alternative formulation based on interpoint distances [5], [6], [45]. Our approach, which allows us to trivially recover $\mathbf{\Theta}$ from our distance-based solution, is developed in detail in Sections IV-A–IV-C and summarized in Fig. 1. In Section IV-D, we prove that our formulation of Problem 3 is equivalent to the DGP [8] for a broad class of manipulators. Finally, in Section IV-E, we show how robots with planar and spherical joints constitute special cases for which our formulation can be trivially simplified.

A. Kinematic Model

Articulated robots are comprised of a series of single-axis revolute joints oriented to provide a useful range of poses in their task spaces. Our goal in this section is to construct a graph representation of such mechanisms that is compatible with the DGP formulation of Problem 2. We achieve this by rigidly attaching a pair of points to the rotation axis of each joint in a manner similar to [6]. As the example in Fig. 2 illustrates, the distances between points corresponding to neighboring joints are invariant to changes in their angles during movement. These distances are key to describing the DOFs of the robot.

Consider the points attached to the rotation axes of neighboring joints, shown in Fig. 2 and labeled u and v. We denote the positions of these points and the orientations of their respective coordinate frames as $\mathbf{p}_u$, $\mathbf{p}_v$, $\mathbf{R}_u$, and $\mathbf{R}_v$, respectively. Furthermore, $\mathbf{p}_{u,v}$ and $\mathbf{R}_{u,v}$ denote the position and orientation of the fixed coordinate frame at v in the rotating coordinate frame at u. The matrix $\mathbf{R}_z(\theta_u)$ rotates $\mathbf{p}_v$ and its child joints about the axis $\hat{\mathbf{z}}$ by the joint angle θ_u. Given the joint angles $\mathbf{\Theta}$, these positions

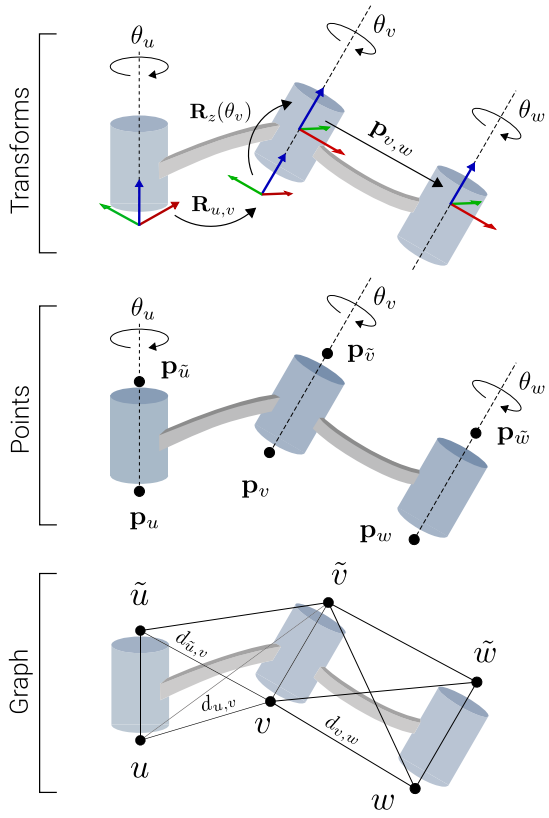

Fig. 2. Visualization of the point placement used to describe a generic linkage of revolute joints and the corresponding graph representation. The top graphic shows the transformations used to obtain the poses of the joint coordinate frames. The middle graphic shows how pairs of points indexed by $(u, \tilde{u})$, $(v, \tilde{v})$, and $(w, \tilde{w})$ are placed along the rotation axis of their respective joints. The bottom graphic shows how the corresponding vertex representations form a graph whose edges are weighted by the known interpoint distances defined by link geometry in (26).

and orientations can be computed recursively as

$$\begin{aligned}\mathbf{R}_v &= \mathbf{R}_u\,\mathbf{R}_z(\theta_u)\,\mathbf{R}_{u,v}\\ \mathbf{p}_v &= \mathbf{p}_u + \mathbf{R}_u\,\mathbf{R}_z(\theta_u)\,\mathbf{p}_{u,v}\;\forall\,(u,v) : v \succ u\end{aligned} \tag{24}$$

where $\succ$ indicates that the joint indexed by u is the parent of the joint indexed by v in the directed graph describing the robot structure.[2] For neighboring joints, $\mathbf{p}_{u,v}$ and $\mathbf{R}_{u,v}$ are determined by the robot model parameterization (e.g., the DH convention [59] or Lie groups [60]). The second pair of points, labeled by $\tilde{u}$ and $\tilde{v}$, is obtained by translation along the axis of rotation of each joint

$$\begin{aligned}\mathbf{p}_{\tilde{u}} &= \mathbf{p}_u + \mathbf{R}_u\hat{\mathbf{z}}\\ \mathbf{p}_{\tilde{v}} &= \mathbf{p}_v + \mathbf{R}_v\hat{\mathbf{z}}\,.\end{aligned} \tag{25}$$

Together, these four points describe the relative position and orientation of the joints' rotation axes. For a given robot, we index the points obtained using (24) and (25) with the set of vertices $V_s \subset V$ of an incomplete graph $G = (V, E)$, where the set of edges E is weighted by interpoint distances that are known *a priori*. These distances describe the overall geometry and DOFs of the robot, and they are invariant to the feasible motions of the robot (i.e., they remain constant in spite of changes to the configuration $\boldsymbol{\Theta}$)

$$\begin{aligned}d_{u,\tilde{u}} &= d_{v,\tilde{v}} = 1\\ d_{u,v} &= \|\mathbf{p}_{u,v}\|\\ d_{u,\tilde{v}} &= \|\mathbf{p}_{u,v} + \mathbf{R}_{u,v}\hat{\mathbf{z}}\|\\ d_{\tilde{u},v} &= \|\mathbf{p}_{u,v} - \hat{\mathbf{z}}\|\\ d_{\tilde{u},\tilde{v}} &= \|\mathbf{p}_{u,v} - \hat{\mathbf{z}} + \mathbf{R}_{u,v}\hat{\mathbf{z}}\|\,.\forall\,(u,v) \in V_s\end{aligned} \tag{26}$$

Depending on the specific link geometry, some identities in (26) may vanish, allowing us to merge identical points and reduce the overall size of the graph.

B. Constraints

In addition to describing the rigid structure of a robot using the distances in (26), we also need to introduce distance constraints that encode features of specific IK problem instances. To that end, this section describes how additional vertices and distances may be used to constrain the end-effector(s) of a robot, implement obstacle avoidance, and enforce joint limits.

1) Base Structure: In order to uniquely specify points with known positions (i.e., end-effectors) in terms of distances, we define the "base vertices" of a robot as $V_b = \{o, x, y, z\} \subset V$, where o is the root or base joint. The elements of V_b are used to form a coordinate frame with vertex o at the origin, as shown in Fig. 3(a). We achieve this by defining the following edge weights:

$$\begin{aligned}d_{o,x} &= d_{o,y} = d_{o,z} = 1\\ d_{x,y} &= d_{x,z} = d_{y,z} = \sqrt{2}.\end{aligned} \tag{27}$$

This base structure is used in the remainder of the section to specify, in terms of distance constraints, end-effector poses and joint limit constraints for links starting at the root. Note that the vertices x and y may be dropped in cases where the root joint angle is not limited to some interval, as this makes the solution set depend only on the distances from the goal points to the root o and to each other. Moreover, we may trivially reduce the graph size by using the points $\mathbf{p}_o$ and $\mathbf{p}_{\tilde{o}}$ attached to the base joint axis instead of the vertices o and z [6].

2) End-Effector Pose: We consider two types of task space goals for an end-effector:

i) a three-DOF position goal ($\mathcal{T}_p = \mathbb{R}^3$);
ii) a five-DOF "direction goal" defined by the position of two distinct points ($\mathcal{T}_d = \mathcal{T}_p \times \mathcal{T}_p$).[3]

In both cases, $\mathbf{w} \in \mathcal{T}$ is encoded as a set of points fixed to the end-effector. These points are indexed by vertices $V_e \subset V$, and their positions are completely determined by the goal $\mathbf{w}$. Thus,

[2]We assume that the graph is a tree whose root is the fixed robot base and whose leaves are the end-effectors; we leave extensions of our formulation to parallel manipulators, which contain loops, for future work.

[3]A full six-DOF pose goal is not supported, as purely distance-geometric constraints cannot prevent reflections of the tool frame: this is equivalent to the assumption that the final joint has no angle limits when that axis is aligned with the final joint (e.g., for a common spherical wrist).

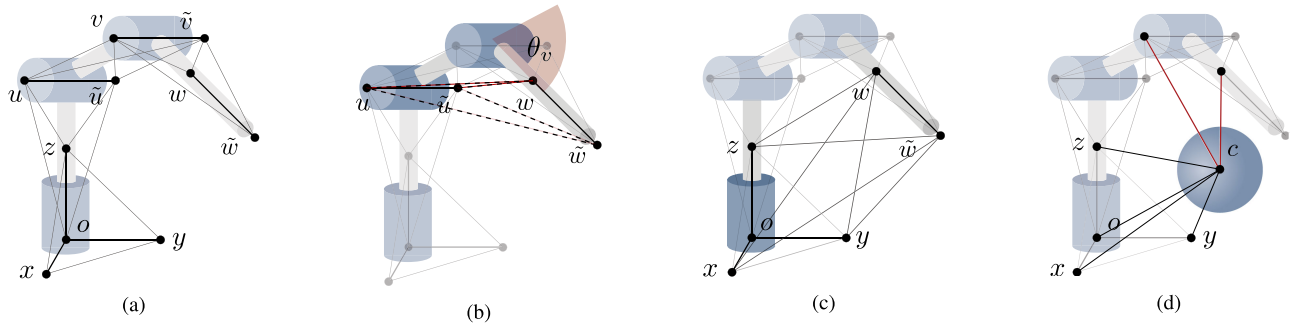

Fig. 3. Visualization of the procedure in Section IV for a three-DOF revolute manipulator with joint limits. (a) Solid black lines represent fixed distances between neighboring joints and between base vertices. (b) Dashed lines correspond to the distances constrained to some interval $[d^-,\ d^+]$ determined by symmetric limits on the joint angle θ_v. (c) Solid black lines represent the distances fixed by setting a desired end-effector pose. (d) Spherical obstacles are represented as vertices in the graph whose position is fixed by defining the distances to the base nodes. The lines drawn in red represent some distances whose lower bounds can be set in order to achieve obstacle avoidance.

an end-effector goal can be enforced by weighting the relevant edges with distances

$$d_{u,v} = \|\mathbf{p}_u - \mathbf{p}_v\|\,, \quad u \in V_e\,, v \in V_b\,. \tag{28}$$

3) Joint limits: Depending on the kinematic structure of the robot, symmetric joint limits can be represented by using distance intervals. In Fig. 3(b), we see that a given joint angle can be represented (up to sign) using up to four distinct distances

$$d^-_{u,w} = \sqrt{d^2_{u,v} + d^2_{v,w} - 2d_{u,v}d_{v,w}\cos(\theta^{\lim}_v)}$$
$$d^+_{u,w} = d_{u,v} + d_{v,w}\,.$$

It follows that joint values can be restricted to symmetric intervals by constraining the distances between vertices assigned to the parent and the child of a particular joint. Generally, limiting one of the aforementioned distances to an interval results in a distinct set of joint angle limits, depending on the particular distance chosen. However, it is important to note that the nature of the distance-based representation may make it difficult to implement arbitrary joint limits, as undesired symmetries may occur in certain ranges.

4) Obstacle Avoidance: We extend our model to incorporate spherical obstacles whose centers are indexed with the set of vertices $V_o \subset V$. The radius of each obstacle is given by the function $\rho : V_o \to \mathbb{R}_+$. Much like the elementary basis vectors in V_b, we can fix each center $\mathbf{p}_c, \forall\, c \in V_o$, in the global reference frame and augment G to include the constant interpoint distances for edges in $V_o \times V_o$ and $V_o \times V_b$:

$$\|\mathbf{p}_i - \mathbf{p}_j\| = d_{i,j} \quad \forall\,(i,j) \in V_o \times V_o \cup V_o \times V_b. \tag{29}$$

Finally, points attached to the joints of a robot (i.e., those indexed by V_s) can be constrained to lie outside of each obstacle

$$\|\mathbf{p}_u - \mathbf{p}_c\| \geq \rho(c) \quad \forall (u,c) \in V_s \times V_o. \tag{30}$$

The radii given by ρ can be inflated to account for the shape and size of the robot's joints or as a conservative safety measure. For robots with long links, auxiliary points indexed by $V_{s'}$ can be easily added between points in V_s for higher precision collision avoidance.

C. Solution Recovery

The remaining edge weights can be determined by completing the resulting partial EDM, and a canonical realization $\mathbf{P}^*$ can be recovered. This result is achieved by identifying the points indexed by V_b with the origin and K elementary basis vectors in $\mathbb{R}^K$, which act as anchors in the solution of the Procrustes procedure [2] discussed in Section III-B. For $K = 3$, since the anchors form a right-handed frame that fully specifies a six-DOF pose, $\mathbf{P}^*$ is unique. In Proposition 1, we prove that $\mathbf{P}^*$ will correspond to a unique feasible configuration $\boldsymbol{\Theta} \in \mathcal{C}$ if the successive joint axes of our robot are *coplanar*. Once $\mathbf{P}^*$ is obtained, we can iteratively recover all the joint values by solving

$$\theta_u = \min_\theta \|\mathbf{R}_u\mathbf{R}_z(\theta)\mathbf{p}_{u,v} - (\mathbf{p}_v - \mathbf{p}_u)\|^2$$
$$+\|\mathbf{R}_u\mathbf{R}_z(\theta)\mathbf{p}_{u,v} + \mathbf{R}_v\hat{\mathbf{z}} - (\mathbf{p}_{\tilde{v}} - \mathbf{p}_u)\|^2\,. \tag{31}$$

This problem can be reduced to finding the roots of a quartic polynomial and, therefore, admits a fast closed-form solution. Alternatively, $\boldsymbol{\Theta}$ can be recovered from (24) and (25) with inverse trigonometric functions. Notably, the optimization formulation of (31) is robust to numerical errors in the calculation of $\mathbf{P}^*$ by Algorithm 1.

D. Equivalence to Distance Geometry

In this section, we prove that the robot models (i.e., the proposed graph descriptions) discussed thus far allow us to solve IK (see Problem 3) by means of the DGP (see Problem 1). The main result is as follows.

Proposition 1 (IK ≡ DGP): Suppose that the kinematic model of Section IV-A describes a robot whose successive joint axes are *coplanar*. Then, the solutions to Problem 1 correspond one-to-one with the solutions to Problem 3. More precisely, if $\mathbf{p}_u, \mathbf{p}_{\tilde{u}}, \mathbf{p}_v$, and $\mathbf{p}_{\tilde{v}}$ are coplanar for all $v \succ u$, then for any end-effector target $\mathbf{w} \in \mathcal{T}$, we have a corresponding DGP encoded in G, and there exists a bijection

$$Q : \mathcal{C}_{\mathbf{w}} \to \mathcal{G} \tag{32}$$

where $\mathcal{C}_{\mathbf{w}} \subset \mathcal{C}$ is the set of configurations achieving $\mathbf{w}$, and $\mathcal{G}$ is the space of all realizations (up to an arbitrary Euclidean transformation) of G.

Proof: We begin by recalling that the presence of base structure constraints [see (27)] in our model allows us to identify "equivalent" realizations of a completion G with a canonical point assignment $\mathbf{P}^*$. We will assume $K = 3$ for the entire proof: $K = 2$ is a special case, which can be simplified by noting that all joints share the same axis of rotation and by removing the "auxiliary" points defined by (25).

For a given robot and $\mathbf{w} \in \mathcal{T}$, the preceding sections described a set of DGP constraints, which we write as the incomplete graph $G = (V, E)$. We will now proceed to prove that there is a bijection between $\mathcal{C}_\mathbf{w}$ and the equivalence classes representing distinct solutions to G (each equivalence class is represented by its canonical solution $\mathbf{P}^*$). It suffices to construct a map Q that is injective and surjective, where Q is simply the iterative procedure described by (24) and (25).

Injective: We will use a proof by contradiction. Suppose $\exists\, \mathbf{\Theta}_1 \neq \mathbf{\Theta}_2 \in \mathcal{C}_\mathbf{w}$ such that $Q(\mathbf{\Theta}_1) = \mathbf{P}^* = Q(\mathbf{\Theta}_2)$. Let u be the vertex label for a joint whose corresponding angle $\mathbf{\Theta}_1^u = \theta_{u_1} \neq \theta_{u_2} = \mathbf{\Theta}_2^u$, but has $\theta_{s_1} = \theta_{s_2}$ for all ancestors s of u. At least one such u is guaranteed to exist because (24) and (25) tell us that the axis points $\mathbf{p}_v$ and $\mathbf{p}_{\tilde{v}}$ of joint $v \succ u$ only depend on θ_u and the positions of all its ancestor joints' points. This gives us

$$\begin{aligned} &\mathbf{p}_u + \mathbf{R}_u \mathbf{R}_z(\theta_{u_1}) \mathbf{p}_{u,v} = \mathbf{p}_u + \mathbf{R}_u \mathbf{R}_z(\theta_{u_2}) \mathbf{p}_{u,v} \\ &\Rightarrow \mathbf{R}_z(\theta_{u_1})^\mathsf{T} \mathbf{R}_z(\theta_{u_2}) = \mathbf{I} \\ &\Rightarrow \theta_{u_1} = \theta_{u_2} \end{aligned} \tag{33}$$

where we have assumed without loss of generality that $\mathbf{p}_{u,v} \neq c\,\hat{\mathbf{z}}$ for some $c \in \mathbb{R}$.[4] This contradicts our premise that $\theta_{u_1} \neq \theta_{u_2}$ and proves that the mapping is injective.

Surjective: We will show that for each $\mathbf{P}^* \in \mathcal{G}$, there exists $\mathbf{\Theta} \in \mathcal{C}_\mathbf{w}$ such that $Q(\mathbf{\Theta}) = \mathbf{P}^* \in \mathcal{G}$. By the definition of $\mathcal{G}$, $\|\mathbf{P}_v^* - \mathbf{P}_u^*\| = \|\mathbf{p}_{u,v}\|$ and $\|\mathbf{P}_v^* - \mathbf{P}_{\tilde{u}}^*\| = \|\mathbf{p}_{u,v} - \hat{\mathbf{z}}\|$ for all $v \succ u$; therefore, we can always find θ_u such that

$$\mathbf{P}_v^* = \mathbf{P}_u^* + \mathbf{R}_u\, \mathbf{R}_z(\theta_u)\, \mathbf{p}_{u,v} \tag{34}$$

as required by (24). Since we have assumed that the points $\mathbf{p}_u, \mathbf{p}_{\tilde{u}}, \mathbf{p}_v$, and $\mathbf{p}_{\tilde{v}}$ are coplanar, the position of $\mathbf{P}_{\tilde{v}}^*$ is uniquely determined by the other three points and, therefore, *must* take the form specified by Q and given by θ_u in (25)

$$\mathbf{P}_{\tilde{v}}^* = \mathbf{P}_v^* + \mathbf{R}_v \hat{\mathbf{z}} = \mathbf{P}_u^* + \mathbf{R}_u\, \mathbf{R}_z(\theta_u)\, \mathbf{p}_{u,v} + \mathbf{R}_v \hat{\mathbf{z}}. \tag{35}$$

If the points were *not* coplanar, the six distance constraints in (26) would only specify their relative positions up to a reflection ambiguity (i.e., there would be two feasible tetrahedra with opposite "chirality" or "handedness"). This situation is illustrated in Fig. 4.

Finally, we address the interval constraints in G, which correspond to joint angle limits of $\mathcal{C}$ and obstacle avoidance in $\mathcal{T}$. We have established the desired bijection $Q : \mathcal{C}_\mathbf{w} \to \mathcal{G}$ for IK problems defined only by equality constraints. Including interval constraints simply limits the space of DGP solutions to $\mathcal{G}' \subset \mathcal{G}$. Since the inverse of a bijection is a bijection, and a subset of the domain of a bijection induces a bijection, we have the desired $Q' : \mathcal{C}_\mathbf{w} \to \mathcal{G}'$, and the proof is complete. □

Proposition 1 establishes that the IK problem for a large class of robots can be formulated as a DGP. Our experiments in Section VI demonstrate that the requirement of coplanar neighboring axes is satisfied by many popular commercial manipulators. Additionally, it is worth noting that planar and spherical manipulators satisfy this requirement by definition.

[4] In the special case where $\mathbf{p}_{u,v} = c\,\hat{\mathbf{z}}$, injectivity can be proved with $\mathbf{p}_{\tilde{v}}$ instead of $\mathbf{p}_v$. If $\mathbf{p}_{\tilde{v}}$ is collinear with $\mathbf{p}_v$, $\mathbf{p}_u$, and $\mathbf{p}_{\tilde{u}}$, then joint v is a rotation around the same axis as joint u, and they can be effectively combined into a single joint.

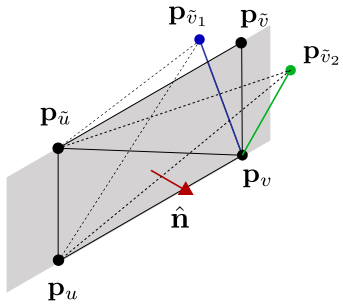

Fig. 4. Illustration of the *chirality* or *handedness* issue that arises for non-coplanar pairs of joint axes $v \succ u$. If $\mathbf{p}_u, \mathbf{p}_{\tilde{u}}, \mathbf{p}_v$, and $\mathbf{p}_{\tilde{v}}$ are coplanar, our distance-geometric approach cannot introduce spurious solutions based on reflections of the true robot geometry. If $\mathbf{p}_{\tilde{v}}$ were replaced with $\mathbf{p}_{\tilde{v}_1}$ as shown, the spurious point $\mathbf{p}_{\tilde{v}_2}$ would also satisfy the distance constraints in (24) because it is a reflection of $\mathbf{p}_{\tilde{v}_1}$ across the plane containing $\mathbf{p}_u, \mathbf{p}_{\tilde{u}}$, and $\mathbf{p}_v$.

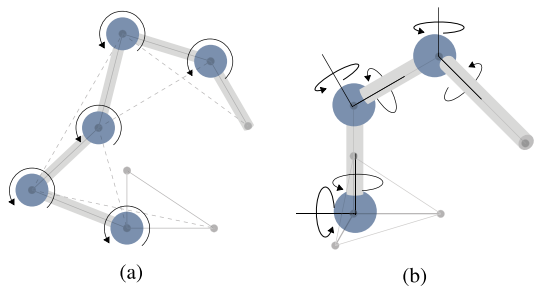

Fig. 5. Visualization of planar and spherical mechanisms. These are two common examples of models for which our IK formulation uses a reduced number of variables.

E. Special Cases

In many cases, the generic DGP formulation of the IK problem presented in this section will result in kinematically redundant points in V_s. One example is points $\mathbf{p}_u$ and $\mathbf{p}_v$ that can be selected to coincide (i.e., $\mathbf{p}_u = \mathbf{p}_v$ for all values of $\mathbf{\Theta}$) because they lie on joint axes $u \succ v$ that intersect. While finding a generic strategy for generating DGP representations with minimal graph sizes is beyond the scope of this article, we can identify two cases of practical importance where this reduction is both trivial and significant.

By reducing the point dimensionality to $K = 2$, we can represent planar mechanisms using a single point per joint, as shown in Fig. 5(a). Without loss of generality, this significantly reduces both the number of points and distances used to describe the IK

problem, resulting in lower overall computation times. For a detailed derivation of this simplified planar problem formulation, see [45].

A similar simplification can be obtained for a class of robots with joints allowing full or partial spherical motion, such as those of the human shoulder and hip. In addition to humanoids, these spherical joints can be found in highly redundant snake-like robots used in applications that include pipe inspection and surgery [61]. As shown in Fig. 5(b), the joints can be represented as two orthogonal revolute joints in the same position, which again allows us to drop the points used to define the rotation axes, while still allowing us to limit the elevation angle and thereby restricting link motions to a spherical cone. The remaining constraints can be trivially derived from those presented for the general case.

V. Algorithm

To summarize so far, in Section IV-A, we derived distance geometric IK formulations for a variety of different robot types. In Sections III-C and III-D, we showed how a Riemannian optimization approach to low-rank matrix completion can be used to efficiently solve instances of the DGP. In this section, we propose an extension to the optimization problem in Section III-D, making it compatible with the distance-geometric IK formulation.

Modifying the matrix $\mathbf{\Omega}$ from (8) to select only those EDM elements corresponding to known invariant distances in $\widetilde{\mathbf{D}}$, we introduce the matrices $\mathbf{\Psi}_-$ and $\mathbf{\Psi}_+$ that select interpoint distances with lower or upper bounds defined by joint limits or obstacle avoidance constraints (i.e., the known elements of $\widetilde{\mathbf{D}}_-$ and $\widetilde{\mathbf{D}}_+$). We can now formulate the DGP as the Riemannian optimization problem

$$\min_{[\mathbf{P}]\in\mathcal{M}} \phi([\mathbf{P}]) \triangleq \frac{1}{2}\left\|\mathbf{\Omega}\odot(\widetilde{\mathbf{D}}-\mathcal{K}\left(\mathbf{P}\mathbf{P}^T\right))\right\|_{\mathrm{F}}^2 + \frac{1}{2}\left\|\max\left\{\mathbf{\Psi}_{\pm}\odot(\pm\widetilde{\mathbf{D}}_{\pm}\mp\mathcal{K}(\mathbf{P}\mathbf{P}^T)),0\right\}\right\|_{\mathrm{F}}^2. \quad (36)$$

The first term in this cost serves to enforce equality constraints on the distance matrix $\mathcal{K}(\mathbf{X})$, representing the constant set of distances defined by robot and task geometry. The second term enforces joint limits and obstacle avoidance constraints by setting either a lower or upper bound on a set of distances related to the joint rotation angle or distances from the obstacle center, with the elementwise $\max$ operator producing a nonzero value when these bounds are violated. Assuming that the joint limits are symmetric, the lower or upper bound on the distances will be implicitly enforced by the link length constraints, meaning that only one bound needs to be explicitly included in (36).

A. Riemannian Trust-Region Algorithm

The quality of the configuration recovered by the reconstruction step outlined in Section IV-C depends on highly accurate solutions to (36). As mentioned in Section III-C, the superlinear convergence guarantee of second-order optimization methods helps to quickly obtain accurate solutions. To maintain this guarantee for the nonisolated minima of (36), we use the second-order RTR algorithm introduced in [62]. Briefly, the trust-region algorithm focuses on sequentially solving the problem

$$\min_{\mathbf{Z}\in\mathcal{H}_{\mathbf{P}}\mathcal{M}} m_{\mathbf{P}}(\mathbf{Z}) \quad \text{s.t.} \quad g_{\mathbf{P}}(\mathbf{Z},\mathbf{Z}) \le \Delta^2 \quad (37)$$

where

$$m_{\mathbf{P}}(\mathbf{Z}) \triangleq \phi(\mathbf{P}) + g_{\mathbf{P}}(\mathbf{Z},\mathrm{grad}_{\mathbf{P}}\phi) + \frac{1}{2}g_{\mathbf{P}}(\mathbf{Z},\mathrm{Hess}_{\mathbf{P}}\phi\,[\mathbf{Z}]). \quad (38)$$

In other words, a quadratic approximation of the model constructed at point $\mathbf{P}$ informs a search for the optimal descent direction $\mathbf{Z}$ within a trust region of radius Δ. Accepting or rejecting a candidate descent direction and updating the trust-region radius is based on the quotient

$$\rho = \frac{\phi(\mathbf{P})-\phi(R_{\mathbf{P}}(\mathbf{Z}))}{m_{\mathbf{P}}(0)-m_{\mathbf{P}}(\mathbf{Z})}. \quad (39)$$

A basic variant of this procedure is formalized in Algorithm 1, where the subproblem in (37) is approximately solved using the truncated conjugate gradient method introduced in [62]. The numerical cost per iteration of this approach was shown in [10] to be $\mathcal{O}(dK+NK+NK^2+K^3)$, where d is the number of known entries in the EDM. Since $K\in[2,3]$ for IK problems, the complexity of Algorithm 1 is linear with respect to the number of points and known distances. Similarly to conventional trust-region methods, the dominant computational bottleneck of RTR is the Hessian calculation, making the additional overhead added by the horizontal projection in (20) comparatively insignificant.

Algorithm 1: Riemannian Trust-Region (RTR).

Input: Initial point $\mathbf{P}_0$
Data: Cost function ϕ
Parameters: $\bar{\Delta}>0$, $\Delta_0\in[0,\bar{\Delta}]$, $\rho'\in[0,\frac{1}{4})$
Result: Solution $\mathbf{P}_N$
for $k=0,1,\ldots N$ **do**
  $\mathbf{Z}_k \leftarrow$ Eq. (37) and Eq. (38) ▷ Compute step
  $\rho \leftarrow$ Eq. (39)
  if $\rho<\frac{1}{4}$ **then** ▷ Update TR radius
    $\Delta_{k+1}\leftarrow\frac{1}{4}\Delta_k$
  else if $\rho>\frac{3}{4}$ *and* $\|\mathbf{Z}_k\|=\Delta_k$ **then**
    $\Delta_{k+1}\leftarrow\min(2\Delta_k,\bar{\Delta})$
  else
    $\Delta_{k+1}\leftarrow\Delta_k$
  end
  if $\rho>\rho'$ **then** ▷ Accept or reject step
    $\mathbf{P}_{k+1}\leftarrow R_{\mathbf{P}}(\mathbf{Z}_k)$ ▷ Retraction (Eq. (23))
  else
    $\mathbf{P}_{k+1}\leftarrow\mathbf{P}_k$
  end
end

B. Bound Smoothing

It is well established that the convergence of local optimization methods depends on the choice of starting point.

Algorithm 2: Inverse Kinematics.

Input: Incomplete graph G, Initial point $\mathbf{P}_0$
Result: Configuration $\mathbf{\Theta}$
Define ϕ from G via (36)
if *not* $\mathbf{P}_0$ **then**
  Get $\mathbf{P}_0$ using bound smoothing (see Section V-B)
end
$\mathbf{P}^* \leftarrow \text{RTR}(\mathbf{P}_0; \phi)$
$\mathbf{P} \leftarrow \text{OrthogonalProcrustes}(\mathbf{P}^*)$
Obtain $\mathbf{\Theta}$ from $\mathbf{P}$ using (31)

The distance-based parameterization of the IK problem has the unique advantage of admitting informed initializations generated via a procedure known as *bound smoothing* [12]. First, we take the known set of distance bounds generated by the robot structure and problem constraints, as described in Section IV, and form the graph G. Next, a bipartite graph is formed with two copies of G, with the vertices of the two graphs connected by edges weighted by their negative respective distance. Finally, the resulting all-pairs shortest path problem is solved using the Floyd–Warshall algorithm in $\mathcal{O}(|V|^3)$ time. For the IK problems analyzed in this article, the computation time of this search is on the order of 1 ms for a simple Python implementation on a laptop computer. The lower bounds on the distances between vertices can now be obtained by taking the shortest path between their representations in different subgraphs, with the upper bounds being the shortest path within one subgraph. An initial guess for the distance matrix, known as a pre-EDM, can then be generated by sampling individual elements within these bounds. Note that this procedure can be applied iteratively to produce even better approximations. Moreover, the computation time can be further reduced through the use of parallelization.

The full algorithm is described in (2). We assume that an incomplete graph G describing the IK problem and an initializing conformation $\mathbf{P}_0$ are provided as input. Alternatively, we may also generate an initialization using the bound smoothing procedure described in the previous section. Next, we solve the local optimization problem using (1) and transform the resulting configuration back to the canonical coordinate system. Finally, we recover the joint angle variables using (31).

VI. Experiments

In this section, we present an analysis of the performance of our method relative to multiple benchmark algorithms in a series of simulation studies. A variety of 2-D and 3-D kinematic models, including commercial manipulators and hyper-redundant kinematic chains and trees, were tested with and without joint angle limits and spherical obstacles. All Python code used in our experiments is freely available in our Git repository.[5]

A. Experimental Methodology

The results for each experiment were obtained with the following procedure.

1) Generate a kinematic model (e.g., a planar robot with links arranged in a perfect binary tree with randomly generated symmetric joint angle limits).
2) Randomly sample an angle configuration $\mathbf{\Theta}_g \in \mathcal{C}$ for this model from a uniform distribution over the joint angle limits.
3) Determine the target position(s) or pose(s) $\mathbf{w} \in \mathcal{T}$ of the end-effector(s) using $\mathbf{\Theta}_g$ and the model's forward kinematics (i.e., $\mathbf{w} = F(\mathbf{\Theta}_g)$).
4) Run each IK algorithm on the problem instance defined by the kinematic model and the goal $\mathbf{w}$ using $\mathbf{\Theta}_0 = \mathbf{0}$ as the initial configuration.
5) Record statistics for each algorithm, including the number of iterations required until convergence, runtime, end-effector error(s), and joint angle limit violations.
6) Repeat steps 2–5 above N times and summarize the statistics.

In all tables and figures, the success rate of an algorithm for a particular experiment was determined as the portion of runs where the solution satisfied all of the following criteria.

1) Joint angle limits were obeyed to within a tolerance of 1% of the bound magnitude.
2) Obstacle avoidance constraints were obeyed to within a tolerance of 0.01 m.
3) The sum of the position errors of the end-effectors was less than 0.01 m.
4) The sum of the rotation errors of the end-effectors was less than 0.01 rad.

The success rates of experiments reported in the tables and waterfall curves correspond to 95% Jeffreys confidence intervals [63]. The statistics on solution error, runtime, and the number of iterations required for convergence that appear in various tables and figures are computed using the entire set of N runs (not just the successful portions). We denote joint angle-limited variants of experiments with a $+$ symbol (e.g., results labeled "6-DOF+" use the same robot as those labeled "6-DOF" but additionally enforce randomly generated joint angle limits).

In all relevant figures, the results from our algorithm are labeled `RTR` for "Riemannian Trust Region." When the bound smoothing procedure of Section V-B is used, `-B` is appended to the label (i.e., `RTR-B`). For each experiment, we compare our approach with a variety of benchmark algorithms from the optimization and IK literature. While we report runtime statistics for many of our experiments, we stress that our selection of baseline algorithms is designed to illustrate a variety of techniques and highlight the unique advantages of our novel distance-geometric formulation. Therefore, we did not choose particularly fast or industrially proven implementations and opted for Python variants of all algorithms. Finally, all experiments were performed on a laptop computer with a 2.9-GHz dual-core Intel Core i5 processor.

B. Benchmark Algorithms

As discussed in Section III, generalized approaches to solving the IK problem most often resort to numerical methods that search for a joint configuration $\mathbf{\Theta} \in \mathcal{C}$ satisfying the defined

[5][Online]. Available: https://github.com/utiasSTARS/GraphIK

constraints. Among the large variety of such approaches, local nonlinear programming is perhaps the most common and versatile. Therefore, we primarily compare our algorithm to the formulation

$$\begin{aligned} \min_{\boldsymbol{\Theta}\in\mathcal{C}} \quad & ||\mathbf{e}\left(F(\boldsymbol{\Theta}),\mathbf{w}\right)||^2 \\ \text{s.t.} \quad & ||\mathbf{p}_i(\boldsymbol{\Theta})-\mathbf{c}_j||^2 \geq l_j^2 \quad \forall\, i\in V_s,\ \ \forall\, j\in V_o \\ & \boldsymbol{\Theta}_{\min}\leq\boldsymbol{\Theta}\leq\boldsymbol{\Theta}_{\max} \end{aligned} \tag{40}$$

where $F(\boldsymbol{\Theta})$ is the forward kinematic mapping and $\mathbf{w}$ is the goal, as defined in Problem 3. The vector-valued function $\mathbf{e}$ in the objective represents an appropriate error for the task space. The inequality constraints serve to enforce obstacle avoidance and joint limits. Note that while the error $\mathbf{e}$ can also be driven to zero with an equality constraint, we have empirically found that incorporating the error in an objective function results in higher success rates—this phenomenon is also reported in [27].

C. Hyper-Redundant and Tree-Like Robots

We begin by analyzing the performance of our algorithm for hyper-redundant and tree-like planar robots. This approach helps to avoid introducing confounding factors in the analysis, as the choice of any particular revolute manipulator opaquely affects the difficulty of IK problems. More importantly, these mechanisms allow us to systematically scale the size and number of constraints of the IK problem by adding joints and introducing multiple end-effectors, while minimizing the number of redundant points and fixed distances as noted in Section IV-E. Since the full pose of a planar end-effector is determined by its position and the position of its parent joint, the error $\mathbf{e}$ in the cost function of the joint angle-based approach in (40) can simply be defined as the difference between the end-effectors' and their parent joints' positions and their goal positions in $\mathbf{w}$. We solve (40) using second-order trust-region methods with similar convergence guarantees to those provided by our algorithm, referring to the unconstrained method as `trust-exact` and the joint angle-constrained method as `trust-constr`. Our open-source code provides an interface for testing this problem formulation with other solvers provided by `scipy.optimize`. In addition to the formulation in (40), we implemented the `FABRIK` [22] heuristic IK solver in Python as an additional benchmark for our experiments. In contrast to generic nonlinear solvers, this is an IK-specific heuristic method tailored to planar and spherical robots.

For the experiments in this section, all algorithms were allowed a total of 2000 iterations and used a numerical tolerance of 10^{-9} for all stopping criteria. Where applicable, the magnitude of the gradient of the cost function or Lagrangian was used as the stopping criterion. Otherwise, the magnitude of the cost function or norm of the variable change in one iteration was used. All other parameters were assigned their default values as provided by the `pymanopt` [64] and `scipy.optimize` libraries [65]. Since the implementations of these benchmark algorithms were not extensively tuned for performance, we place greater emphasis on the number of iterations taken by each algorithm as a more meaningful statistic than runtime in the results to follow. `FABRIK` was allowed a maximum of 2000 full forward and backward iterations per problem instance.

Table I summarizes our results for 6- and 10-DOF planar manipulators of the type shown in Fig. 5(a), with and without joint angle limits. Unsurprisingly, all four algorithms achieve a 100% success rate when no joint angle limits are present. When joint angle limits are introduced, `FABRIK` performs far worse, while the `RTR` algorithm performs similarly to `trust-constr` in terms of success rate and iterations required. Initializing the problem using bound smoothing reduces the number of iterations needed while increasing the success rate of our `RTR-B` algorithm compared to a naive initialization. Curiously, both `trust-constr` and `FABRIK`'s performance improve as the number of DOFs increases. We suspect that the additional DOFs give the heuristic approach of `FABRIK` more time to "steer" away from a difficult initial configuration induced by pose goals. The `trust-constr` algorithm may benefit from more DOFs that can be used to "escape" from local minima or extremely flat regions of the cost function landscape. While we only report on experiments involving *pose* goals in this article, we found that when joint angle limits were introduced, `FABRIK` was much better suited to *position* goals (i.e., without a specified orientation) for the robot end-effector(s).

Table II contains results for experiments involving binary tree-like robots, such as the 6-DOF example with a height of two shown in Fig. 8. The 14- and 30-DOF results correspond to robots with a perfect binary tree kinematic structure of height three and four, respectively. While not as practical as chain-like robots or the revolute manipulators of Section VI-D, these experiments serve to showcase the performance of IK methods on highly kinematically constrained mechanisms, while also scaling naturally to a higher number of DOFs. In all cases, the `RTR` and `RTR-B` algorithms outperform the three benchmark approaches in terms of success rate. When no joint limits are present, the overall difference in success rates is relatively small, with `RTR` and `trust-exact` having a similar number of outer iterations. When joint limits are introduced, the experimental procedure in Section VI-A generates problems within a tighter range around a naive initialization, resulting in higher success rates for all approaches. Due to a high number of constraints, the number of outer iterations for `trust-constr` increases more than tenfold, significantly degrading performance. In contrast, this effect does not occur for `RTR` and `RTR-B`, where the number of iterations remains considerably lower while a similar increase in success rate is observed. The box-and-whiskers plots in Fig. 6 summarize the position error statistics for each algorithm over *all* runs in the constrained case, including those runs that did *not* qualify as a success. The waterfall curves in Fig. 7 display the success rate as a function of an increasing position error tolerance, demonstrating that the higher success rate of our algorithm is maintained for different accuracy requirements.

Mean runtimes of both `RTR` and `RTR-B` across all planar experiments remain below 0.1 s, with the 30-DOF tree-like robot unsurprisingly resulting in the most computationally intensive problems. The runtime for `FABRIK` across all planar experiments was 3.4 s, while the local solvers had a mean runtime of 10.2 s. Both of these sets of runtime statistics are influenced by

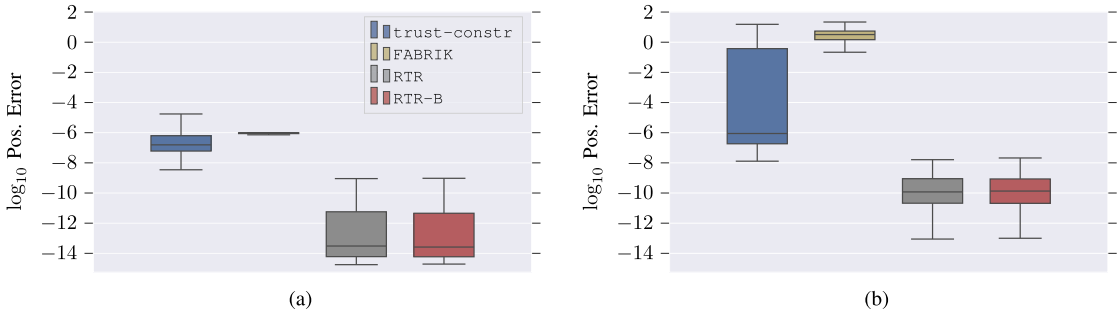

Fig. 6. Box-and-whiskers plots summarizing end-effector position error over 1000 experiments with planar binary tree robots with joint angle limits. (a) 14-DOF+. (b) 30-DOF+.

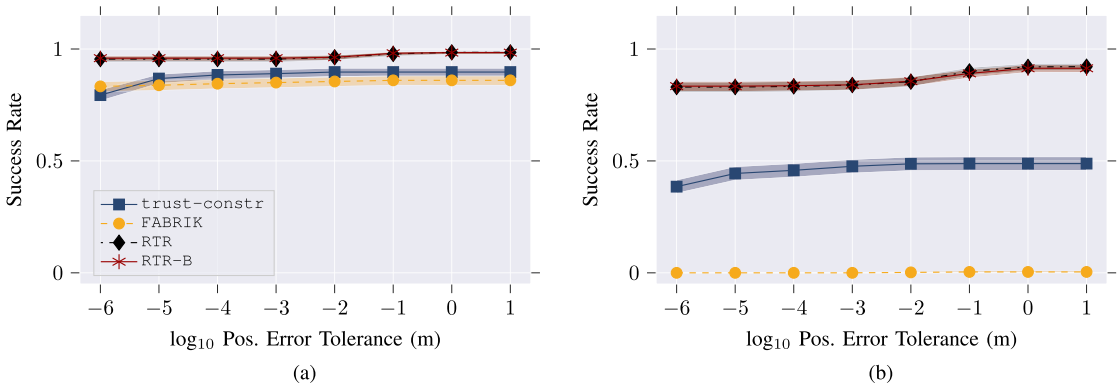

Fig. 7. Waterfall curves of success rate versus position error tolerance for 1000 experiments with planar tree robots with joint angle limits. The shaded regions are 95% Jeffreys confidence intervals centered on the solid lines. The rotation error tolerance is fixed at 0.01 rad. (a) 14-DOF+. (b) 30-DOF+.

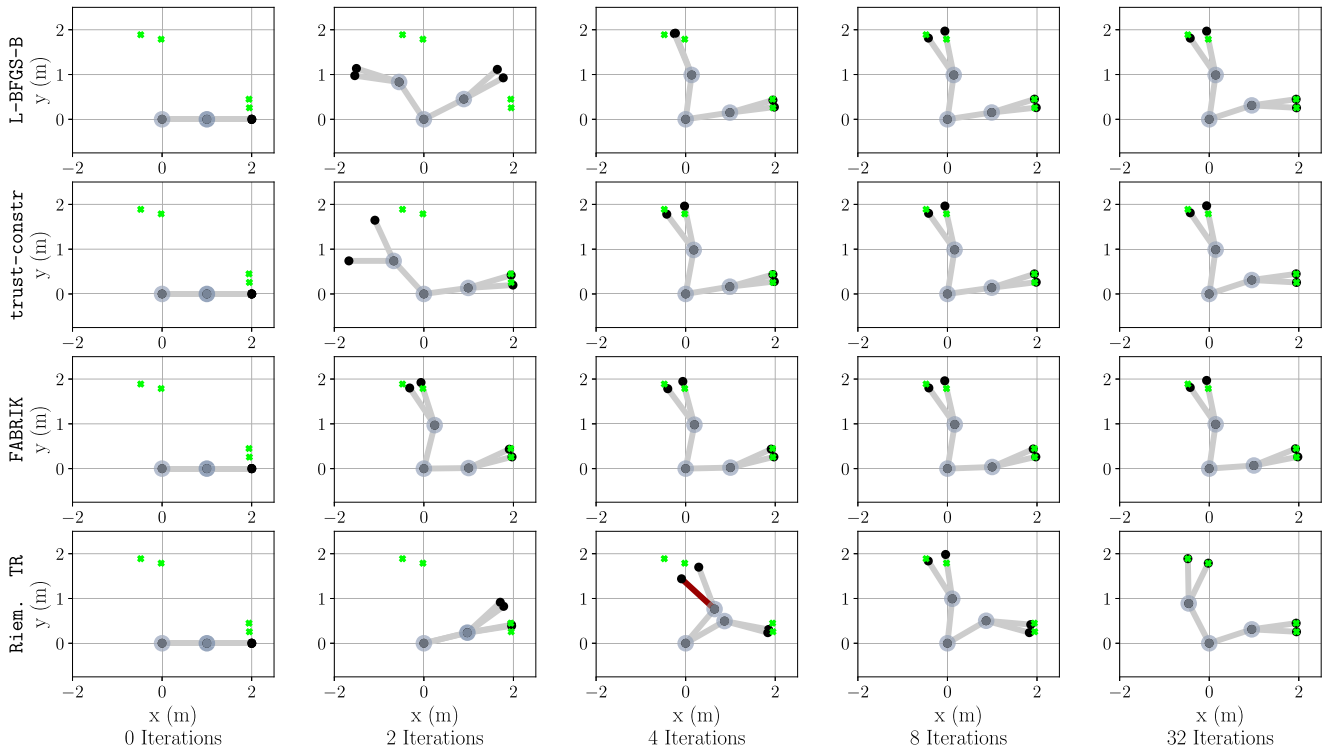

Fig. 8. Convergence of various algorithms on a 6-DOF binary tree robots with joint angle limits. `FABRIK` and the angle-parameterized `L-BFGS-B` and `trust-constr` all converge to a local minimum, whereas `Riem. TR` is able to converge to the global minimum from the same initial condition ($\boldsymbol{\Theta} = \mathbf{0}$). Note that `FABRIK` quickly converges but is unable to accurately reach any of the four end-effector targets. The red link in the `RTR` solution after four iterations indicates that the link's parent joint is violating its joint angle limits.

TABLE I
RESULTS FOR PLANAR CHAIN MANIPULATORS OVER 1000 RANDOM EXPERIMENTS WITH POSE GOALS

Method	`trust-exact/constr`		`FABRIK`		`RTR`			`RTR-B`		
	Success (%)	Iter. μ (σ)	Success (%)	Iter. μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)
6-DOF	100.0 ± 0.1	26 (5)	100.0 ± 0.1	10 (19)	100.0 ± 0.1	16 (2)	2.28 (0.39)	100.0 ± 0.1	9 (2)	1.54 (0.37)
6-DOF+	72.0 ± 2.8	35 (14)	36.0 ± 3.0	1333 (916)	91.0 ± 1.7	32 (22)	5.24 (2.83)	$\mathbf{98.0 \pm 0.9}$	24 (17)	5.02 (3.01)
10-DOF	100.0 ± 0.1	28 (2)	100.0 ± 0.1	9 (13)	100.0 ± 0.1	20 (2)	3.27 (0.53)	100.0 ± 0.1	10 (2)	2.25 (0.43)
10-DOF+	97.0 ± 1.0	43 (54)	59.0 ± 3.0	856 (971)	88.0 ± 2.0	74 (101)	14.6 (1.15)	$\mathbf{98.0 \pm 0.8}$	23 (46)	5.47 (3.85)

+ indicates joint angle limits.

TABLE II
RESULTS FOR PLANAR TREE MANIPULATORS OVER 1000 RANDOM EXPERIMENTS WITH POSE GOALS

Method	`trust-exact/constr`		`FABRIK`		`RTR`			`RTR-B`		
	Success (%)	Iter. μ (σ)	Success (%)	Iter. μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)
14-DOF	54.0 ± 3.1	14 (4)	56.0 ± 3.1	949 (964)	64.0 ± 3.0	20 (4)	4.39 (1.00)	$\mathbf{67.0 \pm 2.9}$	9 (3)	4.12 (1.11)
14-DOF+	90.0 ± 1.9	189 (423)	85.0 ± 2.2	463 (716)	$\mathbf{96.0 \pm 1.2}$	18 (4)	4.98 (1.40)	96.0 ± 1.2	8 (2)	4.79 (1.76)
30-DOF	16.0 ± 2.3	23 (7)	12.0 ± 2.0	1848 (479)	$\mathbf{20.0 \pm 2.5}$	34 (10)	20.75 (8.52)	18.0 ± 2.4	18 (10)	45.25 (32.38)
30-DOF+	49.0 ± 3.1	818 (830)	0.0 ± 0.3	2000 (0)	$\mathbf{85.0 \pm 2.2}$	36 (19)	31.64 (17.95)	85.0 ± 2.2	20 (15)	43.98 (32.62)

+ indicates joint angle limits.

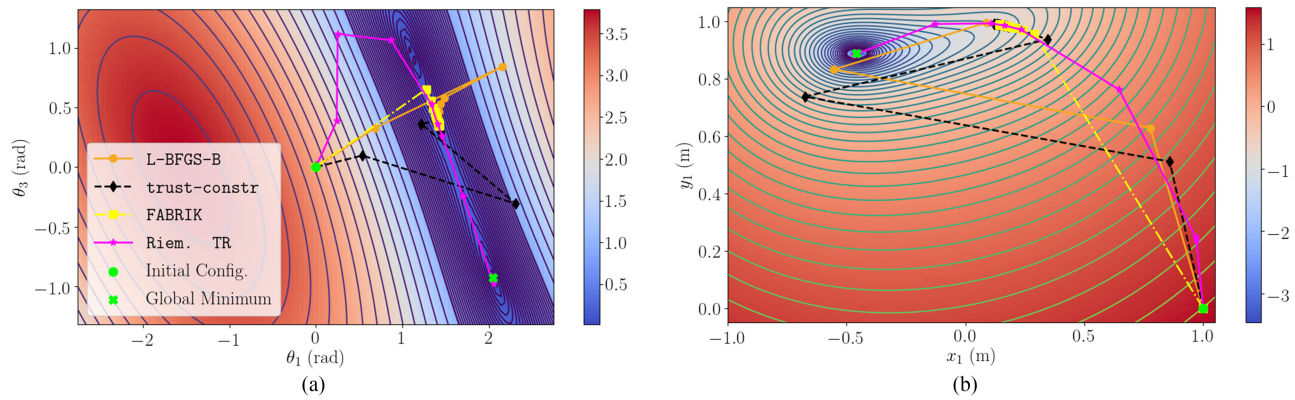

Fig. 9. Contour maps of different cost function components overlaid with solver trajectories for the IK problem instance depicted in Fig. 8. The angle θ_1 is the angle of the joint connected to the root and pointing towards the top left of corner of the plot of `Riem. TR` after 32 iterations in Fig. 8. Angles θ_3 and θ_4 are the angles of the two child links of the link actuated by θ_1. In (b), $x_1 = \cos\theta_1$ and $y_1 = \sin\theta_1$ are the coordinates of the point actuated by θ_1, and the cost function is the quartic terms of (9) involving x_1 and y_1. This distance-geometric cost function is very well posed, and the Riemannian solver follows its contours to the global minimum. The other three methods attempt to minimize the ill-conditioned cost function in (a) and return a suboptimal solution. (a) L_2 error for the end-effector actuated by θ_1 and θ_3. (b) Base-10 logarithm of distance-geometric cost involving the joint position which is actuated by θ_1.

the large proportion of unsuccessful runs that required all 2000 allowed iterations before terminating. However, since we cannot guarantee that the local algorithms were provided with equally well-tuned implementations of core subroutines (e.g., Hessian computations for the second-order trust-region solvers), we urge our readers to treat the statistics on iterations reported in Tables I and II as more qualitative indicators of performance.

To illustrate the optimization procedure and help elucidate the relatively superior performance of our method on branching tree-like robots, we conducted a simple empirical analysis on one of the many problem instances where `RTR` outperformed the benchmark algorithms. Fig. 8 shows the convergence of four solvers with the same initial condition on a sample low-dimensional IK problem involving a 6-DOF binary planar tree robot with symmetric joint angle limits and end-effector position goals. Only our algorithm, `RTR`, is able to find the global minimum. The algorithms all perform similarly for the first eight iterations, but the three competitors are unable to escape from the same local minimum. The difference in behavior is explained by Fig. 9, which compares contour maps of different cost function terms used by the algorithms, overlaid with the progress of each algorithm across iterations. Fig. 9(a) illustrates the Euclidean distance of the end-effector controlled by θ_1 and θ_3 and its goal position, which is used in the cost function of `L-BFGS-B` and `trust-constr`. In contrast, the contour map in Fig. 9(b) is the logarithm of the quartic function containing the terms in the distance-geometric cost function of (9) involving the position of the joint actuated by θ_1. The angular cost function is ill-conditioned, leading the algorithms that minimize it to converge to the local minimum of Fig. 8, whereas `RTR` minimizes the distance-geometric cost of Fig. 9(b) and quickly converges to the global minimum, which has a large and well-conditioned basin of convergence. In spite of its simplicity, this toy problem illustrates the behavioral differences of the algorithms in a state space with low enough dimension to visualize clearly.

TABLE III
RESULTS FOR REVOLUTE CHAIN MANIPULATORS OVER 2000 RANDOM EXPERIMENTS WITH POSE GOALS

Method	`trust-exact/constr`		`RTR`			`RTR-B`		
	Success (%)	Iter. μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)	Success (%)	Iter. μ (σ)	Runtime [ms] μ (σ)
UR10	90.0 ± 1.3	12 (7)	87 ± 1.0	364 (523)	205.8 (264.1)	$\mathbf{95.0 \pm 1.0}$	**319** (**493**)	198.4 (239.3)
UR10+	63.0 ± 2.1	36 (19)	$\mathbf{77.0 \pm 1.8}$	**364** (**523**)	206.6 (264.2)	66.0 ± 2.0	251 (457)	138.2 (193.0)
KUKA	100.0 ± 0.1	20 (6)	100.0 ± 0.2	317 (373)	242.8 (211.1)	100.0 ± 0.1	315 (386)	268.3 (220.0)
KUKA+	87.0 ± 1.5	48 (18)	82.0 ± 1.7	734 (771)	432.4 (395.1)	$\mathbf{89 \pm 1.3}$	**506** (**660**)	386.8 (374.8)
LWA4P	$\mathbf{100.0 \pm 0.2}$	**20** (**17**)	89.0 ± 1.4	513 (624)	416.6 (542.4)	90.0 ± 1.3	503 (614)	290.0 (330.1)
LWA4P+	77.0 ± 1.8	46 (25)	$\mathbf{87.0 \pm 1.5}$	**435** (**569**)	246.7 (282.2)	81 ± 1.7	324 (482)	201.1 (219.7)
LWA4D	$\mathbf{100.0 \pm 0.1}$	**24** (**17**)	99.0 ± 0.5	868 (747)	969.9 (878.0)	97.0 ± 0.7	713 (699)	895.6 (897.1)
LWA4D+	$\mathbf{96.0 \pm 0.8}$	**47** (**17**)	91.0 ± 1.3	867 (769)	900.2 (874.1)	90.0 ± 1.3	825 (758)	991.2 (943.8)

+ indicates joint angle limits.

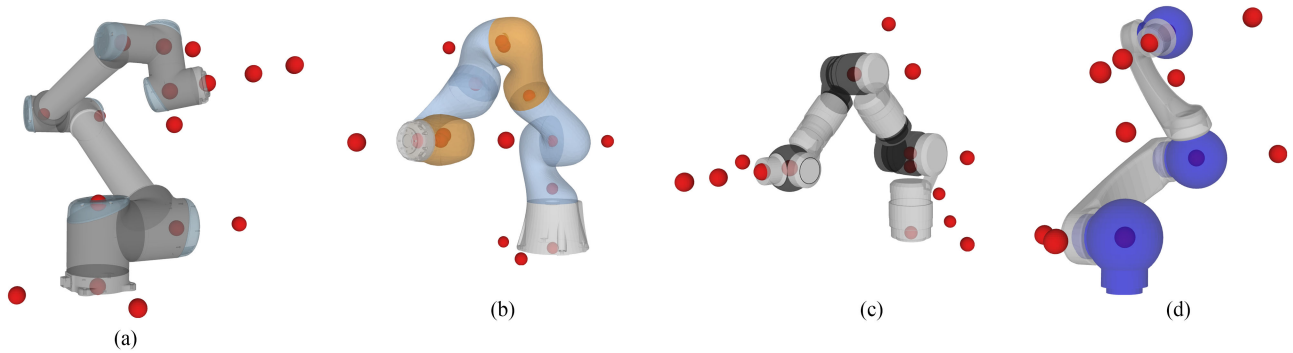

Fig. 10. Points in the distance-based models of the commercial manipulators used in our experiments. Note that the distances between pairs of points on individual rotation axes have been reduced for clarity. (a) UR10. (b) KUKA-IIWA. (c) Schunk-LWA4D. (d) Schunk-LWA4P.

D. Revolute Manipulators

Next, Table III compares the performance of our algorithm and the trust-region benchmark algorithms on 3-D robots with revolute joints and within an unconstrained workspace. These problems are of greater practical interest than the planar results in Section VI-C and showcase the expressiveness of our problem formulation. All experiments are conducted for the Universal Robots UR10, KUKA IIWA, Schunk LWA4D, and Schunk LWA4P manipulators shown in Fig. 10. For these robots, the distance-geometric IK formulation derived using the procedure described in Section IV-A results in points that always overlap (i.e., have a fixed distance of zero). While these points could be "merged" to reduce the graph size—thereby improving overall performance—we used the generic models for transparency.

In terms of success rate, our algorithm outperforms `trust-exact` and `trust-constr` on the UR10 and KUKA IIWA manipulators with and without joint angle limits, as well as on the Schunk LWA4P with joint limits. The bound smoothing procedure used to initialize `RTR-B` reduces the overall number of iterations in all cases, but has a variable effect on the success rate. Both `RTR` and `RTR-B` require a significantly larger number of iterations to converge compared to the planar case, while `trust-*` remains in a similar range to that observed in Table I. We can partially attribute this to the iteration complexity discussed in Section V, which is increased by the unfavorably high ratio of points to DOFs in the kinematic models of these mechanisms. Again, this effect could be mitigated in future work by removing overlapping points from the kinematic models, reducing the overall number of variables.

The box-and-whiskers plots in Fig. 11 summarize the position error statistics for each algorithm over *all* runs with the UR10 manipulator, with and without joint limits. In both cases, the `trust-*` algorithms converge to significantly lower cost function values. This suggests that the highly variable magnitudes of known distances may cause numerical issues in the gradient for our algorithms, causing early termination. We suspect this issue can be avoided by using a weighting matrix to regularize elements of the cost function, as shown in [42]. The waterfall curves in Fig. 12 corroborate these findings, showing that the success rate of our algorithm drops as the position error tolerance decreases, and suggesting that decreasing the gradient termination criteria may increase accuracy at the cost of increased computation time.

The mean runtime of both `RTR` and `RTR-B` remains below 1.0 s for all robot models, reaching the lowest mean runtimes of less than 0.2 s on the UR10. In the same instance, the `trust-*` algorithms have a slightly higher mean runtime of 0.3 s. We observe the highest computation times for our algorithms with the 7-DOF Schunk LWA4D, with mean runtimes slightly below 1.0 s. While the `trust-*` methods exhibit similarly worse performance with a mean runtime of 0.5 s, the overall increase in runtime is smaller due to the number of variables only increasing from six to seven.

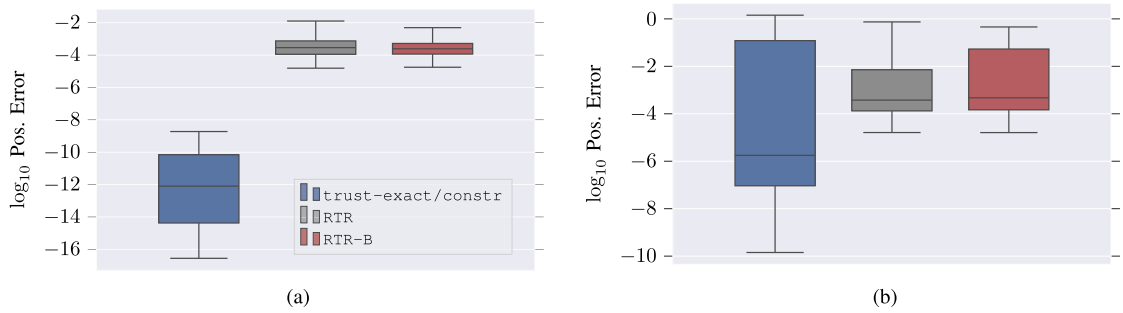

Fig. 11. Box-and-whiskers plots summarizing end-effector position error over 2000 experiments with the UR10 manipulator, (a) without and (b) with joint angle limits. (a) UR10. (b) UR10+.

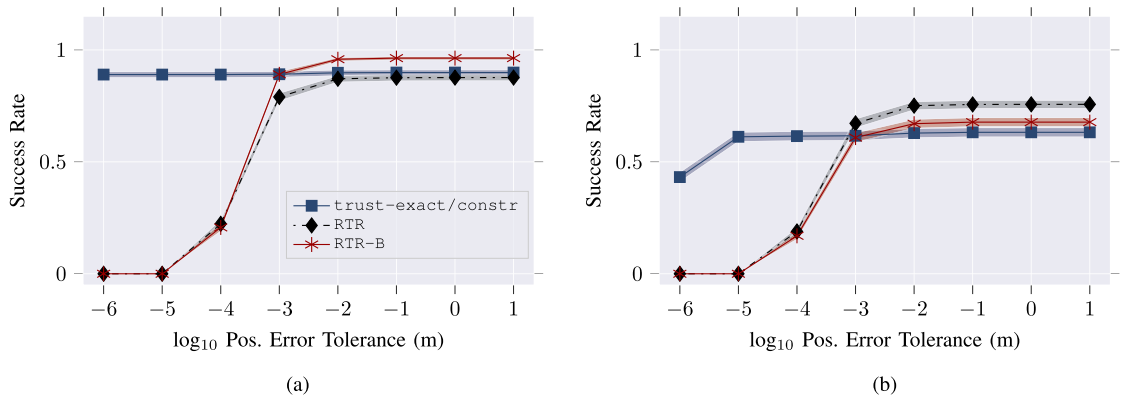

Fig. 12. Waterfall curves of success rate versus position error tolerance for 2000 experiments with the UR10 manipulator, without (a) and with (b) joint angle limits. The shaded regions are 95% Jeffreys confidence intervals centered on the solid lines. The rotation error tolerance is fixed at 0.01 rad. (a) UR10. (b) UR10+.

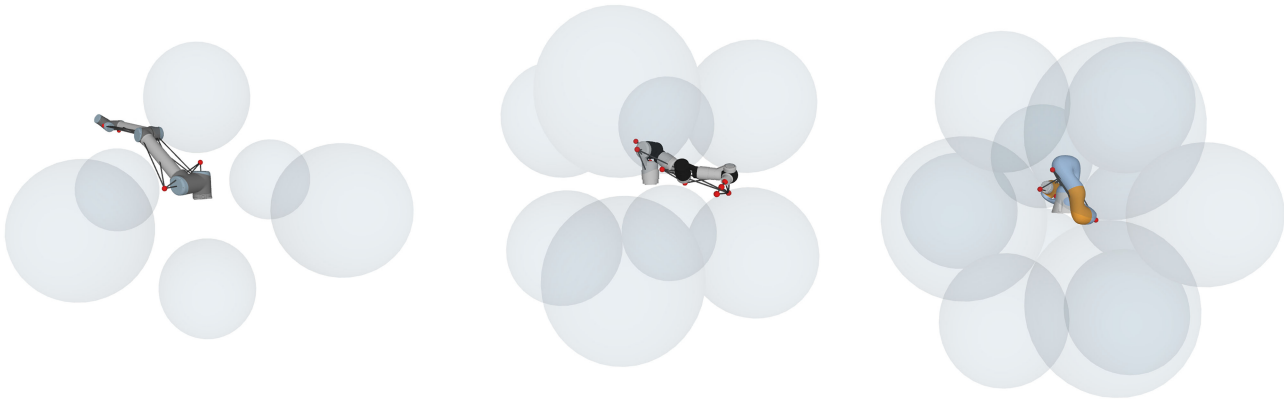

Fig. 13. Selection of robot and environment combinations used to generate IK problems in Section VI-E. (a) Universal Robotics UR10 (*octahedron*). (b) Schunk LWA4D (*cube*). (c) KUKA-IIWA (*icosahedron*).

E. Obstacle Avoidance

Finally, we analyze the performance of our algorithm on revolute manipulators in environments with a varying number of spherical obstacles, as shown in Fig. 13. For each environment and robot pair, 3000 IK problems were randomly generated, some of which may not be solvable (i.e., no collision-free solutions exist). For experiments performed in this section, we chose $\mathbf{e} = \log(\mathbf{T}(\boldsymbol{\Theta})^{-1}\mathbf{T}_{\text{goal}})$ as the error in (40), which is the vector of exponential coordinates describing the screw motion between the current pose and the goal pose of the end-effector. Furthermore, the FK and Jacobian computations are carried out using the popular *product of exponentials* approach [60], avoiding trigonometric identities inherent to the DH parameterization. We solve the problem in (40) using sequential quadratic programming, namely, the `SLSQP` solver implemented in the `scipy` library. Due to its speed and accuracy, this method is a popular choice for nonlinear programming solutions to IK [27]. We empirically determined that a maximum of 200 iterations and an objective

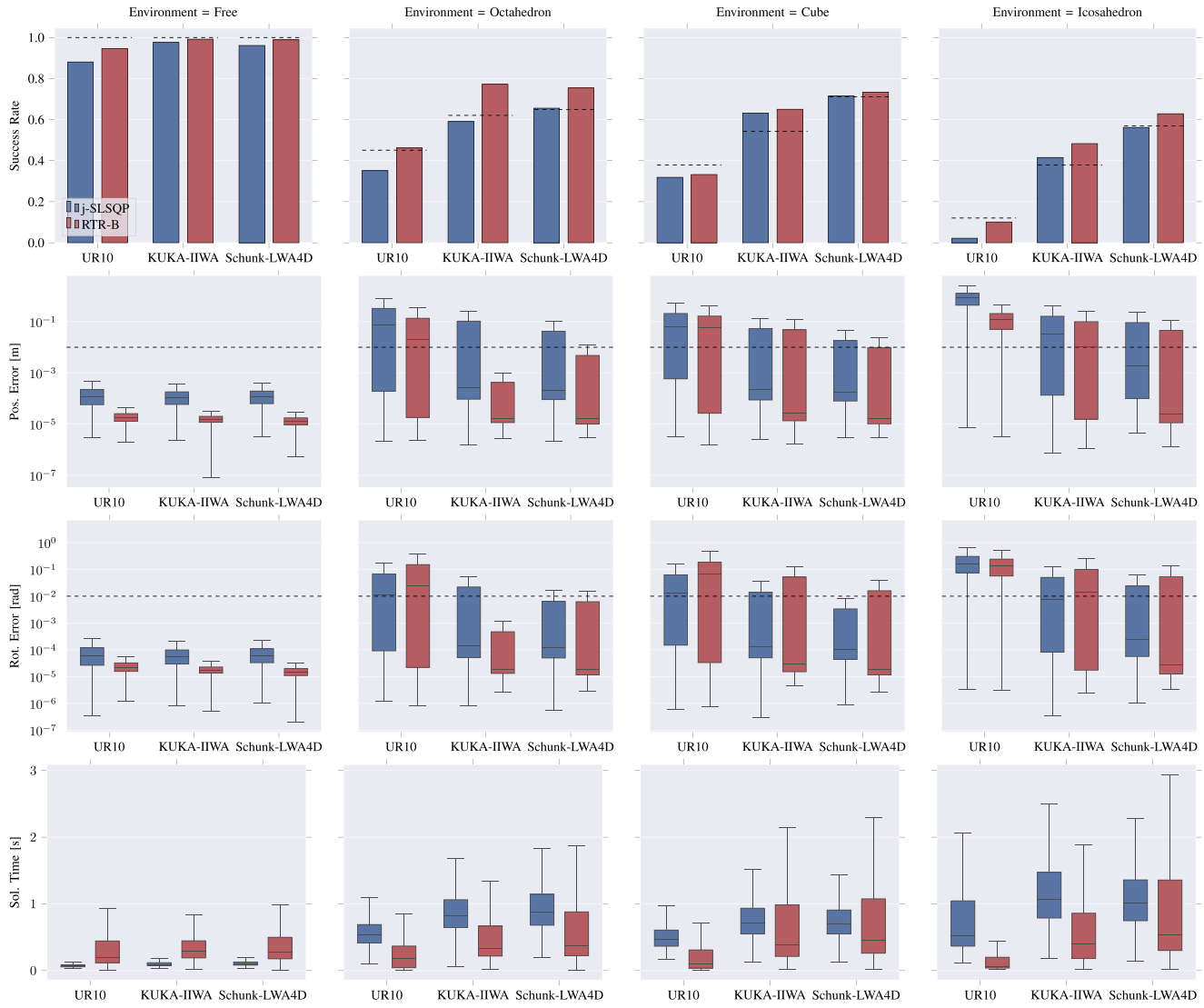

Fig. 14. Experimental results comparing our method with local optimization. Each column contains the success rates, position errors, rotation errors, and solution times for 3000 randomly generated problems in the titular environment. The success rate in the top row is measured relative to the total number of generated problems, many of which are infeasible. The upper bound defining success in each box-and-whiskers plot is shown as a dashed line. The dashed lines in the top row indicate the lower bound on the number of feasible problems for each robot–environment pair.

function value of 10^{-7} were effective stopping criteria for our experiments.

The results of our evaluation are shown in Fig. 14. Each column represents a different fixed set of obstacles in the manipulators' environment. The first column contains results for obstacle-free problems, while the remaining columns use spherical obstacles placed on the vertices of an octahedron, cube, and icosahedron, respectively. The top row compares the success rate of `RTR-B` and `SLSQP`, with dashed lines indicating the portion of problems that are known to be feasible (i.e., the configuration used to generate the problem is not in collision). The box-and-whiskers plots in the middle two rows describe the distribution of position and rotation errors, with the thresholds for success (0.01 m and 0.01 rad) drawn as dashed lines. Finally, the bottom row compares the distribution of solution times.

In terms of success rate, our algorithm outperforms `SLSQP` in all experiments. The position and rotation error distributions displayed in the box-and-whiskers plots reveal that this is due to `RTR-B` providing solutions with lower position and rotation error on average. The performance gap is particularly large for the UR10, which both algorithms struggled most with in all environments. Finally, the runtime for our Riemannian solver is expectedly higher than for `SLSQP` in the obstacle-free case, as seen in previous experiments. However, the addition of obstacles leads to significantly faster relative performance for `RTR-B` on all the manipulators tested. More importantly, we note that the runtime of `SLSQP` exhibits a more significant relative increase than `RTR-B` when obstacles are introduced. We suspect that this is due to the nonlinear mapping between joint angles and obstacle locations that makes the problem significantly more difficult to solve in configuration space. In contrast, the effect

is avoided by our distance-based approach because collision avoidance constraints are treated in the same way as structural and joint limit constraints.

VII. Conclusion

In this article, we presented a novel and elegant procedure for formulating many IK problems in the language of distance geometry. This distance-geometric perspective on IK allowed us to leverage powerful low-rank matrix completion methods, resulting in an algorithm (`RTR-*`) that can efficiently compute IK solutions for a variety of robots using Riemannian optimization. Our experiments showed that that `RTR-*` outperforms competing algorithms in terms of success rate and the number of iterations on IK problems for robots with multiple end-effectors, both with and without the inclusion of joint limits. We also demonstrated the feasibility of this approach to solve IK for commercial manipulators, achieving success rates competitive with both comparable [32] and conventional [60] angle-based methods. Notably, we observed that our algorithm performs significantly better than a conventional method, both in terms of success rate and runtime, when the IK problem requires finding configurations that are not in collision with obstacles (modeled as spheres). Overall, these experimental results indicate that a distance-geometric approach is particularly advantageous when a large number of workspace constraints are present. We believe our algorithm provides a valuable benchmark for IK solvers, as well as a good starting point for future research into distance-geometric formulations of this problem.

A. Limitations

Avoiding reflections in the solution set of certain problems remains a core challenge when using a purely distance-based IK approach. As noted in Proposition 1, this spells out an important limitation of our formulation: its inability to handle revolute manipulators with noncoplanar consecutive axes of rotation. This also restricts the capacity of our formulation to represent joint angle limits to those that are symmetrical about $\mathbf{\Theta}_0 = \mathbf{0}$ (i.e., of the form $[-\theta_{\lim},\ \theta_{\lim}]$). At the cost of some of the theoretical foundations laid out in Sections III and IV-D, it is possible to address these ambiguities by extending our formulation to include cross products (allowing "handedness" to be expressed). For example, solutions that include reflections for noncoplanar joints u and v in Fig. 2 could be removed by taking $\mathbf{c} = (\mathbf{p}_{\bar{u}} - \mathbf{p}_u) \times (\mathbf{p}_{\bar{v}} - \mathbf{p}_v)$ and constraining the sign of the dot product $\mathbf{c} \cdot (\mathbf{p}_v - \mathbf{p}_u)$, thereby only allowing one of the two possible relative orientations satisfying distance constraints. Similarly, a nonsymmetric joint limit on v can be obtained by taking $\mathbf{a} = (\mathbf{p}_v - \mathbf{p}_u) \times (\mathbf{p}_{\bar{v}} - \mathbf{p}_v)$, $\mathbf{b} = (\mathbf{p}_w - \mathbf{p}_u) \times (\mathbf{p}_{\bar{v}} - \mathbf{p}_v)$ and constraining the sign of $\mathbf{a} \cdot \mathbf{b}$. Since adding these constraints would require a lengthy and detailed deviation from the purely distance-geometric view of IK developed herein, we leave their characterization as future work.

B. Future Work

We have identified several exciting research directions for the distance-geometric IK formulation presented in this article. Our algorithm utilizes a Riemannian optimization-based solution because it is fast, can easily be extended (e.g., to include obstacles or other cost terms), and avoids problems associated with redundant DOFs. However, we believe a thorough exploration of the vast body of literature on distance geometry may yield other effective approaches or insights into our particular problem structure. Our hope is that further study of approaches such as semidefinite programming relaxations for EDM completion will help to yield deep theoretical insight into the geometric structure of IK and its relationship with other problems in distance geometry. This will permit us to compare the runtime of our algorithm against a greater variety of IK solvers, including complex algorithms like `TRAC-IK` that utilize multiple methods in parallel [27]. Additionally, we are eager to develop an optimized version of our algorithm in a fast compiled language such as C^{++}. We believe that the distance-geometric IK formulation described herein provides a strong mathematical basis for future research in kinematics, motion planning, and control.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. New York, NY, USA: Springer, 2010.

[2] I. Dokmanic, R. Parhizkar, J. Ranieri, and M. Vetterli, "Euclidean distance matrices: Essential theory, algorithms and applications," *IEEE Signal Process. Mag.*, vol. 32, no. 6, pp. 12–30, Nov. 2015.

[3] J. G. de Jalón, "Twenty-five years of natural coordinates," *Multibody Syst. Dyn.*, vol. 18, no. 1, pp. 15–33, Aug. 2007.

[4] F. Blanchini, G. Fenu, G. Giordano, and F. A. Pellegrino, "A convex programming approach to the inverse kinematics problem for manipulators under constraints," *Eur. J. Control*, vol. 33, pp. 11–23, Jan. 2017.

[5] T. Le Naour, N. Courty, and S. Gibet, "Kinematics in the metric space," *Comput. Graph.*, vol. 84, pp. 13–23, 2019.

[6] J. M. Porta, L. Ros, and F. Thomas, "Inverse kinematics by distance matrix completion," in *Proc. 12th Int. Workshop Comput. Kinematics*, 2005, Art. no. 61-CK2005.

[7] L. Han and L. Rudolph, "Inverse kinematics for a serial chain with joints under distance constraints," in *Proc. Robot.: Sci. Syst. Conf.*, 2006, pp. 177–184.

[8] L. Liberti, C. Lavor, N. Maculan, and A. Mucherino, "Euclidean distance geometry and applications," *SIAM Rev.*, vol. 56, no. 1, pp. 3–69, Jan. 2014.

[9] L. T. Nguyen, J. Kim, and B. Shim, "Low-rank matrix completion: A contemporary survey," *IEEE Access*, vol. 7, pp. 94 215–94 237, 2019.

[10] B. Mishra, G. Meyer, and R. Sepulchre, "Low-rank optimization for distance matrix completion," in *Proc. IEEE Conf. Decis. Control Eur. Control Conf.*, Dec. 2011, pp. 4455–4460.

[11] M. Journée, F. Bach, P.-A. Absil, and R. Sepulchre, "Low-rank optimization on the cone of positive semidefinite matrices," *SIAM J. Optim.*, vol. 20, no. 5, pp. 2327–2351, Jan. 2010.

[12] T. F. Havel, "Distance geometry: Theory, algorithms, and chemical applications," in *Encyclopedia of Computational Chemistry*. Hoboken, NJ, USA: Wiley, Apr. 2002, pp. 723–742.

[13] J. Angeles, G. Hommel, and P. Kovács, *Computational Kinematics*, vol. 28. New York, NY, USA: Springer, 2013.

[14] A. Aristidou, J. Lasenby, Y. Chrysanthou, and A. Shamir, "Inverse kinematics techniques in computer graphics: A survey," *Comput. Graph. Forum*, vol. 37, no. 6, pp. 35–58, Sep. 2018.

[15] H.-Y. Lee and C.-G. Liang, "A new vector theory for the analysis of spatial mechanisms," *Mech. Mach. Theory*, vol. 23, no. 3, pp. 209–217, 1988.

[16] D. Manocha and J. F. Canny, "Efficient inverse kinematics for general 6R manipulators," *IEEE Trans. Robot. Autom.*, vol. 10, no. 5, pp. 648–657, Oct. 1994.

[17] M. L. Husty, M. Pfurner, and H.-P. Schröcker, "A new and efficient algorithm for the inverse kinematics of a general serial 6R manipulator," *Mech. Mach. Theory*, vol. 42, no. 1, pp. 66–81, 2007.

[18] R. Diankov, "Automated construction of robotic manipulation programs," Ph.D. dissertation, The Robotics Institute, Carnegie Mellon Univ., Pittsburgh, PA, USA, Sep. 2010.

[19] B. Kenwright, "Inverse kinematics-cyclic coordinate descent (CCD)," *J. Graph. Tools*, vol. 16, no. 4, pp. 177–217, 2012.

[20] R. Muller-Cajar and R. Mukundan, "Triangulation—A new algorithm for inverse kinematics," in *Proc. Int. Conf. Image Vis. Comput.*, Dec. 2007, pp. 181–186.
[21] Li Han and Lee Rudolph, "A unified geometric approach for inverse kinematics of a spatial chain with spherical joints," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2007, pp. 4420–4427.
[22] A. Aristidou and J. Lasenby, "FABRIK: A fast, iterative solver for the inverse kinematics problem," *Graph. Models*, vol. 73, no. 5, pp. 243–260, Sep. 2011.
[23] A. Aristidou, Y. Chrysanthou, and J. Lasenby, "Extending FABRIK with model constraints," *Comput. Animation Virtual Worlds*, vol. 27, no. 1, pp. 35–57, Jan. 2016.
[24] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Trans. Math. Softw.*, vol. 23, no. 4, pp. 550–560, Dec. 1997.
[25] J. Schulman *et al.*, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, Aug. 2014.
[26] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
[27] P. Beeson and B. Ames, "TRAC-IK: An open-source library for improved solving of generic inverse kinematics," in *Proc. IEEE Int. Conf. Humanoid Robots (Humanoids)*, 2015, pp. 928–935.
[28] L. Sciavicco and B. Siciliano, "Coordinate transformation: A solution algorithm for one class of robots," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-16, no. 4, pp. 550–559, Jul. 1986.
[29] S. R. Buss and J.-S. Kim, "Selectively damped least squares for inverse kinematics," *J. Graph. Tools*, vol. 10, pp. 37–49, 2005.
[30] Y. Nakamura, H. Hanafusa, and T. Yoshikawa, "Task-priority based redundancy control of robot manipulators," *Int. J. Robot. Res.*, vol. 6, no. 2, pp. 3–15, 1987.
[31] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Hoboken, NJ, USA: Wiley, 2005.
[32] K. Erleben and S. Andrews, "Solving inverse kinematics using exact hessian matrices," *Comput. Graph.*, vol. 78, pp. 1–11, Feb. 2019.
[33] H. Dai, G. Izatt, and R. Tedrake, "Global inverse kinematics via mixed-integer convex optimization," *Int. J. Robot. Res.*, vol. 38, no. 12/13, pp. 1420–1441, Oct. 2019.
[34] T. Yenamandra, F. Bernard, J. Wang, F. Mueller, and C. Theobalt, "Convex optimisation for inverse kinematics," *Proc. Int. Conf. 3D Vis.*, Sep. 2019, pp. 318–327.
[35] F. Blanchini, G. Fenu, G. Giordano, and F. A. Pellegrino, "Inverse kinematics by means of convex programming: Some developments," in *Proc. IEEE Int. Conf. Autom. Sci. Eng.*, Aug. 2015, pp. 515–520.
[36] Y. Ding, N. Krislock, J. Qian, and H. Wolkowicz, "Sensor network localization, Euclidean distance matrix completions, and graph realization," *Optim. Eng.*, vol. 11, no. 1, pp. 45–66, Feb. 2010.
[37] M. A. Cox and T. F. Cox, "Multidimensional scaling," in *Handbook of Data Visualization*. New York, NY, USA: Springer, 2008, pp. 315–347.
[38] L. Liberti, C. Lavor, and N. Maculan, "A branch-and-prune algorithm for the molecular distance geometry problem," *Int. Trans. Oper. Res.*, vol. 15, pp. 1–7, 2008.
[39] P. Biswas, T.-C. Lian, T.-C. Wang, and Y. Ye, "Semidefinite programming based algorithms for sensor network localization," *ACM Trans. Sens. Netw.*, vol. 2, no. 2, pp. 188–220, 2006.
[40] N.-H. Z. Leung and K.-C. Toh, "An SDP-based divide-and-conquer algorithm for large-scale noisy anchor-free graph realization," *SIAM J. Sci. Comput.*, vol. 31, no. 6, pp. 4351–4372, 2010.
[41] B. Vandereycken, "Low-rank matrix completion by Riemannian optimization—Extended version," *SIAM J. Optim.*, vol. 23, no. 2, pp. 1214–1236, Sep. 2012.
[42] L. T. Nguyen, J. Kim, S. Kim, and B. Shim, "Localization of IoT networks via low-rank matrix completion," *IEEE Trans. Commun.*, vol. 67, no. 8, pp. 5833–5847, Aug. 2019.
[43] J. M. Porta, N. Rojas, and F. Thomas, "Distance geometry in active structures," in *Mechatronics for Cultural Heritage and Civil Engineering*, vol. 92. New York, NY, USA: Springer, 2018, pp. 115–136.
[44] J. Porta, L. Ros, F. Thomas, and C. Torras, "A branch-and-prune solver for distance constraints," *IEEE Trans. Robot.*, vol. 21, no. 2, pp. 176–187, Apr. 2005.
[45] F. Marić, M. Giamou, S. Khoubyarian, I. Petrović, and J. Kelly, "Inverse kinematics for serial kinematic chains via sum of squares optimization," in *Proc. IEEE Int. Conf. Robot. Autom.*, Aug. 2020, pp. 7101–7107.
[46] P. A. Parrilo, "Semidefinite programming relaxations for semialgebraic problems," *Math. Prog.*, vol. 96, no. 2, pp. 293–320, May 2003.
[47] J. B. Lasserre, "Global optimization with polynomials and the problem of moments," *SIAM J. Optim.*, vol. 11, no. 3, pp. 796–817, Jan. 2001.
[48] B. Hendrickson, "Conditions for unique graph realizations," *SIAM J. Comput.*, vol. 21, no. 1, pp. 65–84, Feb. 1992.
[49] M. J. Sippl and H. A. Scheraga, "Cayley–Menger coordinates," *Proc. Nat. Acad. Sci.*, vol. 83, no. 8, pp. 2283–2287, Apr. 1986.
[50] A. Y. Alfakih, A. Khandani, and H. Wolkowicz, "Solving Euclidean distance matrix completion problems via semidefinite programming," *Comput. Optim. Appl.*, vol. 12, nos. 1–3, pp. 13–30, 1999.
[51] A. M.-C. So and Y. Ye, "Theory of semidefinite programming for sensor network localization," *Math. Prog.*, vol. 109, no. 2-3, pp. 367–384, Jan. 2007.
[52] S. Burer and R. D. Monteiro, "Local minima and convergence in low-rank semidefinite programming," *Math. Program.*, vol. 103, no. 3, pp. 427–444, Dec. 2004.
[53] H. Fang and D. P. O'Leary, "Euclidean distance matrix completion problems," *Optim. Methods Softw.*, vol. 27, nos. 4/5, pp. 695–717, Oct. 2012.
[54] D. I. Chu, H. C. Brown, and M. T. Chu, "On least squares Euclidean distance matrix approximation and completion," Dept. Math., North Carolina State Univ., Raleigh, NC, USA, Tech. Rep., 2003.
[55] B. A. Pearlmutter, "Fast exact multiplication by the hessian," *Neural Comput.*, vol. 6, no. 1, pp. 147–160, Jan. 1994.
[56] P.-A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*. Princeton, NJ, USA: Princeton Univ. Press, 2009.
[57] N. Boumal and P.-A. Absil, "Low-rank matrix completion via preconditioned optimization on the Grassmann manifold," *Linear Algebra Appl.*, vol. 475, pp. 200–239, 2015.
[58] K. Wei, J.-F. Cai, T. F. Chan, and S. Leung, "Guarantees of Riemannian optimization for low rank matrix recovery," *SIAM J. Matrix Anal. Appl.*, vol. 37, no. 3, Apr. 2016.
[59] R. S. Hartenberg and J. Denavit, "A kinematic notation for lower pair mechanisms based on matrices," *J. Appl. Mech.*, vol. 77, no. 2, pp. 215–221, 1955.
[60] K. M. Lynch and F. C. Park, *Modern Robotics*. Cambridge, U.K.: Cambridge Univ. Press, 2017.
[61] H. Ananthanarayanan and R. Ordóñez, "Real-time inverse kinematics of (2n+1) DOF hyper-redundant manipulator arm via a combined numerical and analytical approach," *Mech. Mach. Theory*, vol. 91, pp. 209–226, Sep. 2015.
[62] P.-A. Absil, C. G. Baker, and K. A. Gallivan, "Trust-region methods on Riemannian manifolds," *Found. Comput. Math.*, vol. 7, no. 3, pp. 303–330, 2007.
[63] T. Cai, "One-sided confidence intervals in discrete distributions," *J. Statist. Plan. Inference*, vol. 131, no. 1, pp. 63–88, Apr. 2005.
[64] J. Townsend, N. Koep, and S. Weichwald, "Pymanopt: A python toolbox for optimization on manifolds using automatic differentiation," *J. Mach. Learn. Res.*, vol. 17, no. 1, pp. 4755–4759, 2016.
[65] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, no. 3, pp. 261–272, 2020.

Filip Marić received the B.Sc. and M.Sc. degrees in electrical engineering and information technology from the Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb, Croatia, in 2015 and 2017, respectively. He is currently working toward the Ph.D. degree with the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto, Toronto, ON, Canada, jointly with the Laboratory for Autonomous Systems and Mobile Robotics, University of Zagreb.

His research interests include applying concepts from differential geometry to kinematics and motion planning in robotics.

Matthew Giamou received the B.A.Sc. degree in engineering science from the University of Toronto, Toronto, ON, Canada, in 2015, and the M.Sc. degree in aeronautics and astronautics from the Massachusetts Institute of Technology, Cambridge, MA, USA, in 2017. He is currently working toward the Ph.D. degree with the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto, where he is investigating convex relaxations for various challenging state estimation and planning problems.

His research interests include multirobot systems and the integration of learned and classical models for robot perception.

Mr. Giamou is a Vector Institute Postgraduate affiliate and a Royal Bank of Canada Graduate Fellow.

Adam W. Hall received the B.Sc.Eng. degree in engineering physics and the B.Sc. degree in chemistry from Queen's University, Kingston, ON, Canada, in 2014. He is currently working towards Ph.D. degree from the University of Toronto, Toronto, ON.

He has worked in the space robotics industry. He is currently a member of both the Space and Terrestrial Autonomous Robotic Systems Laboratory and the Dynamic Systems Laboratory, University of Toronto. He is exploring how to apply classical notions of safety to learning-based control problems.

Soroush Khoubyarian is working toward the B.A.Sc. degree in engineering science with the University of Toronto, Toronto, ON, Canada.

He was a Student with the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto Institute for Aerospace Studies, Toronto, in the summers of 2019 and 2020, where he worked on convex optimization methods for sensor calibration and manipulation.

Ivan Petrović received the Master of Science and Ph.D. degrees in electrical engineering from the Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb, Croatia, in 1990 and 1998, respectively.

He is currently a Professor and the Head of the Laboratory for Autonomous Systems and Mobile Robotics, Faculty of Electrical Engineering and Computing, University of Zagreb. He has authored or coauthored about 60 journal and 200 conference papers. His current research interests include advanced control and estimation techniques and their application in autonomous systems and robotics.

Dr. Petrović is a Full Member of the Croatian Academy of Engineering and the Chair of the Technical Committee on Robotics of the International Federation of Automatic Control.

Jonathan Kelly received the Ph.D. degree in computer science from the University of Southern California, Los Angeles, CA, USA, in 2011.

From 2011 to 2013, he was a Postdoctoral Fellow with the Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, Cambridge, MA, USA. He is currently the Dean's Catalyst Professor and the Director of the Space and Terrestrial Autonomous Robotic Systems Laboratory, University of Toronto Institute for Aerospace Studies, Toronto, ON, Canada. He holds the Tier II Canada Research Chair in Collaborative Robotics. His research interests include perception, planning, and learning for interactive robotic systems.

Dr. Kelly is a Vector Institute Faculty Affiliate and a Schwartz Reisman Institute for Technology and Society Faculty Affiliate.